Faculty of Computer Science
and Information Technology

Through the Eye

VR Simulation of Retinal Diseases with AI-Chatbot

Submitted by:

Omar Ehab	**42210220**
Nada Ayman	**42210205**
Toka Ashraf	**42210068**
Norhan Essam	**42210253**
Alaa Maher	**42210074**
Amir Ayman	**42210195**

A dissertation submitted in partial fulfillment of the requirements for the degree of Bachelor of computer science and information technology

Supervised by:

Dr. Belal Badawy

EGYPT, 2026

Committee Report

We certify we have read this graduation project report as examining committee, examine the student in its content and that in our opinion it its adequate as a project document for "*Through the Eye*".

Chairman:
Name:
Signature:
Date: / /20

Supervisor:
Name:
Signature:
Date: / /20

Examiner:
Name:
Signature:
Date: / /20

Intellectual Property Right Declaration

This is to declare that the work under the supervision of Dr. Belal Badawy having title "*Through the Eye*" carried out in partial fulfillment of the requirements of Bachelor of Science in Computer Science is the sole property of Ahram Canadian University and the respective supervisor. It is protected under the intellectual property right laws and conventions. It can only be considered/ used for purposes like extension for further enhancement, product development, adoption for commercial/organizational usage, etc. with the permission of the University and respective supervisor. This above statement applies to all students and faculty members.

Names:

#	ID	Name
1	42210220	Omar Ehab Mohamed
2	42210205	Nada Ayman
3	42210068	Toka Ashraf
4	42210074	Alaa Maher
5	42210253	Norhan Essam
6	42210195	Amir Ayman

Supervised by: Dr. Belal Badawy

Anti-Plagiarism Declaration

This is to declare that the above publication produced under the supervision of Dr. Belal Badawy having title "*Through the Eye*" is the sole contribution of the author(s) and no part hereof has been reproduced illegally (cut and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Names:

#	ID	Name
1	42210220	Omar Ehab Mohamed
2	42210205	Nada Ayman
3	42210068	Toka Ashraf
4	42210074	Alaa Maher
5	42210253	Norhan Essam
6	42210195	Amir Ayman

Abstract

Visual impairment caused by retinal diseases represents a major global health challenge, affecting millions of individuals and significantly impacting quality of life. Despite this prevalence, existing educational and awareness tools often fail to convey the real visual experience of patients, relying instead on static images or textual descriptions that limit understanding and empathy.

This project, *Through the Eye*, presents a web-based Virtual Reality (VR) system designed to simulate the visual distortions associated with a wide range of retinal and vision-related diseases. The system supports the simulation of twelve conditions, including age-related macular degeneration, retinitis pigmentosa, Stargardt disease, pathological myopia, choroideremia, macular pucker, acute central serous chorioretinopathy, hypertensive retinopathy, and multiple cataract types. The platform utilizes WebXR and Three.js to deliver immersive, browser-accessible 3D simulations without requiring native VR installations or specialized hardware. Medically inspired shader-based post-processing techniques are employed to realistically reproduce visual impairments such as blurring, tunnel vision, glare, and central vision loss.

In addition to visual simulation, the platform integrates an AI-powered chatbot to provide real-time support, allowing users to ask questions related to disease symptoms, progression, and prevention. This integration transforms the experience from a passive visualization into an interactive tool aimed at enhancing awareness and empathy among medical students, healthcare professionals, and the general public.

The proposed system prioritizes accessibility, medical accuracy, and educational value while maintaining performance constraints required for VR comfort. By bridging the gap between theoretical knowledge and sensory experience, *Through the Eye* demonstrates the potential of web-based VR and AI technologies as effective tools for health education and awareness.

Contents

Committee Report 2

Intellectual Property Right Declaration 3

Anti-Plagiarism Declaration 4

Abstract 5

List of Figures 9

Chapter 1: Introduction 12

1.1 Overview 12

1.2 Motivation 12

1.3 Objective 13

1.4 Aim 13

1.5 Scope 13

1.6 General constraints 14

1.7 Organization of the dissertation 15

1.8 Problem Definition 16

Chapter 2: Background and Previous work 18

2.1 Background 18

2.1.1 Virtual Reality (VR) 18

2.1.2 Frontend Concepts: 22

2.1.3 Backend Concepts: 25

2.1.4 Artificial Intelligence (AI): 28

2.2 Previous work 28

2.2.1 Academic and Clinical Research Platforms 30

2.2.2 Public Awareness VR Tools 30

2.2.3 Light Web-Based Tools 32

2.2.4 Identified Gaps in Previous Work 33

Chapter 3: Planning and Analysis 35

3.1 Planning 35

3.1.1 Feasibility study and estimated cost 35

3.1.2 Gantt Chart 39
3.2 Analysis and Limitations of Existing System 40
3.2.1 Analysis of Existing Systems 40
3.2.2 Limitations of Existing Systems 42
3.3 Need for New System 42
3.4 Analysis of new system 43
3.4.1 User Requirements 43
3.4.2 System Requirements 45
3.4.3 Domain Requirements 47
3.4.4 Functional Requirements 48
3.4.5 Non-Functional Requirements 49
3.5 Advantages of the New System 50
3.6 User Characteristics 52
Chapter 4: Design 55
4.1 Design and Implementation Constraints 55
4.1.1 Technological Constraints 55
4.1.2 API and AI Constraints 56
4.1.3 Medical Accuracy Constraints 56
4.1.4 Performance Constraints 57
4.1.5 Platform & Deployment Constraints 57
4.1.6 Development Constraints 58
4.1.7 Ethical and Legal Constraints 58
Ethical Use of AI 58
Intellectual Property Constraints 58
User Privacy & Data Protection 58
4.2 Assumptions and Dependencies 58
4.2.1 Assumptions 58
4.2.2 Dependencies 61
4.3 Design of Database ERD 64

4.3.1 Entity Relationship Diagram 64
4.3.2 Mapping of ERD 65
4.4 Class diagram 66
4.5 Use case Diagram 67
4.5.1 Primary Use-case diagram 67
4.5.2 Use-case scenarios 68
4.6 Activity Diagram 83
4.7 Sequence Diagram 90
4.8 State Diagram 97
References 99

List of Figures

Figure 1: First HMD developed by Ivan Sutherland in 1968 18
Figure 2: Generic web-based XR architecture 19
Figure 3: Classic VR system architecture 19
Figure 4: web-Based VR System Architecture of the Proposed Platform 21
Figure 5: Frontend Runtime Architecture of the Web-Based VR System 23
Figure 6: Comparison of Common Backend Architectural Styles 25
Figure 7: Monolithic Backend Architecture of the Proposed System 26
Figure 8: XREye Visual Impairment Simulation 30
Figure 9: NEI "See What I See" (National Eye Institute) 31
Figure 10: EmpathEyes Vision Simulator 32
Figure 11: Example of a Light Web-Based Distortion Tool (VisionSim) 33
Figure 12: Gantt Chart 39
Figure 13: Sign Up (User) 83
Figure 14: Login, Logout (User, Admin) 84
Figure 15: View Diseases Educational Info (User) 85
Figure 16:Start, Navigate, and control VR simulation (User) 86
Figure 17:Interact with AI Chatbot (User) 87
Figure 18: Add Disease (Admin) 88
Figure 19: Manage accounts (Admin) 89
Figure 20: Signup (User) 90
Figure 21: Login (User) 91
Figure 22: Login (Admin) 91
Figure 23: Logout (User) 92
Figure 24: Logout (Admin) 92
Figure 25: View diseases List (User) 93
Figure 26: View diseases details (User) 93
Figure 27: Interact with AI chatbot (User) 94
Figure 28: Start VR Simulation 94
Figure 29: Manage accounts (Admin) 95
Figure 30: Add Disease (Admin) 96
Figure 31: Simulation 97
Figure 32: Chatbot 98

List of Tables

Table 1: estimated cost 38
Table 2: User Requirements 43
Table 3: System Requirements 45
Table 4: Domain Requirements 47
Table 5: Regular User Functional Requirements 48
Table 6: Admin Functional Requirements 49
Table 7: Non-Functional Requirements 49
Table 8: UC-01 68
Table 9: UC-02 70
Table 10: UC-03 72
Table 11: UC-04 73
Table 12: UC-05 74
Table 13: UC-06 75
Table 14: UC-07 76
Table 15: UC-08 77
Table 16: UC-09 79
Table 17: UC-10 80
Table 18: UC-11 81

Chapter 1 Introduction

Chapter 1: Introduction

1.1 Overview

"*Through the Eye*" is an immersive Virtual Reality (VR) simulation system designed to replicate the visual experience of individuals suffering from retinal diseases. The project integrates WebXR, Three.js, and real-time shader-based visual distortion techniques to simulate medical conditions such as Retinitis Pigmentosa, Age-related Macular Degeneration (AMD), Cataracts, and other retinal disorders.

Additionally, the system is supported by an AI-powered chatbot, developed using the OpenAI API, which provides accurate explanations, preventive guidance, and interactive educational support.

By delivering medically accurate visual distortion effects within a 3D VR environment, the project aims to enhance empathy, awareness, and clinical understanding among medical students, professionals, and the general public.

1.2 Motivation

Visual impairment affects more than 2.2 billion people worldwide. Despite this prevalence, traditional learning methods—such as diagrams, lectures, and textbook images do not effectively convey how patients actually see through diseased eyes. This creates a significant empathy gap between clinicians and patients, and limits public awareness regarding symptoms and early detection.

Existing awareness tools are limited due to one or more of the following:

- They are 2D, non-interactive, or purely illustrative.
- They require complex installations.
- They do not allow users to explore disease progression dynamically.
- They lack real-time intelligent educational support.

This project is motivated by addressing these gaps through a web-based, accessible, interactive VR simulation that transforms theoretical knowledge into a practical sensory experience.

1.3 Objective

- **Simulate visual distortions** associated with retinal diseases using medically validated post-processing shader effects.
- **Develop a web-based VR environment** using WebXR and Three.js that runs on standard browsers with no installation required.
- **Integrate an AI chatbot** capable of answering medical questions, explaining symptoms, and providing prevention guidelines.
- **Enhance empathy and educational understanding** by allowing users to “experience” diseases rather than only reading about them.
- **Ensure cross-platform compatibility**, allowing use on desktop browsers, VR headsets, or standard devices.

1.4 Aim

The aim of this project is to develop an accessible VR-based educational tool that bridges the gap between theoretical ophthalmology knowledge and lived patient experience. By offering real-time disease simulation combined with intelligent medical guidance, the system aims to improve empathy, support medical education, and increase public awareness regarding visual impairments.

1.5 Scope

The scope of this project covers the end-to-end development of the "*Through the Eye*" web application. This includes:

- Research: A literature review on VR in ophthalmology and empathy-driven simulations.
- VR Development: The design and implementation of a WebXR system using Three.js and WebGL shaders to simulate visual distortions like blur, glare, tunnel vision, and color loss for specific retinal diseases.
- AI Integration: The implementation of an AI chatbot using the OpenAI API to handle natural language queries and provide medically accurate responses.
- System Architecture: Building a React.js frontend for interactivity and an ASP.NET Core MVC backend with a SQL Server database to manage data and API calls.

- Testing and Documentation: The project scope includes full system integration, testing, and the preparation of final documentation and presentation materials.

1.6 General constraints

The project is subject to several constraints:

- **Technological Constraint:** The project is strictly browser-based, built on WebXR, Three.js, and WebGL shaders. This constrains the graphical fidelity and performance to what standard web browsers can support, as opposed to more powerful native VR applications (e.g., built on Unity or Unreal).
- **API Dependency:** The AI chatbot's functionality is entirely dependent on the OpenAI API. This introduces constraints related to:
    - The need for a stable internet connection for real-time interaction.
    - Potential API call costs.
    - Possible latency in chatbot responses.
    - Reliance on a third-party service's availability and policies.
- **Compatibility:** The system must maintain cross-browser compatibility. This is a significant technical challenge, as different web browsers (Chrome, Firefox, Safari, Edge) have varying levels of support and performance for WebXR and WebGL.
- **Medical Accuracy:** The visual simulations are not merely artistic; they must be medically accurate representations of retinal diseases. This requires that the shader development be strictly guided by ophthalmological research, constraining purely creative or technical design decisions.
- **Cost-Effectiveness:** The project aims to be a cost-effective solution, which guides the choice of technologies (web-based over proprietary) and likely limits the use of expensive assets or software licenses.

1.7 Organization of the dissertation

This dissertation is organized as follows:

- **Chapter 1: Introduction -** This chapter provides a comprehensive overview of the "*Through the Eye*" project, states the motivation behind it, and defines its primary objectives, aim, and scope. It also outlines the general constraints encountered and presents this roadmap for the dissertation's organization.
- **Chapter 2: Background and Previous work -** This chapter reviews the foundational concepts relevant to the project, including VR in medical education and retinal disease simulation. It then analyzes existing systems and previous research (previous work) to establish a baseline and identify gaps that this project addresses.
- **Chapter 3: Planning and Analysis -** This chapter details the project management and requirements engineering phase. It includes the project plan, a feasibility study with estimated costs, and a Gantt chart. It also analyzes the limitations of existing systems, justifies the need for the new system, and defines its comprehensive requirements, including user, system, domain, functional, and non-functional requirements.
- **Chapter 4: Design -** This chapter presents the complete architectural and system design. It discusses design constraints, assumptions, dependencies, and risk management strategies. The core of this chapter is the detailed system modeling, which includes the database ERD, class diagrams, use case diagrams and scenarios, activity diagrams, sequence diagrams, and state diagrams.
- **Chapter 5: Implementation -** This chapter describes the practical development phase of the project. It outlines the final software architecture, showcases the design and implementation of the user interface (UI), and presents the results of the development, along with a discussion of the outcomes.
- **Chapter 6: Testing -** This chapter focuses on the verification and validation of the system. It details the testing methodologies employed,

providing specific examples of unit testing for individual components and integration testing to ensure all parts of the system function correctly together.

The dissertation concludes with a summary of the project's achievements, limitations, and potential directions for Future work. A complete list of References cited throughout the document is provided at the end.

1.8 Problem Definition

Despite the global prevalence of visual impairments, current awareness and educational tools remain largely theoretical and fail to convey the sensory reality of these conditions. Existing approaches such as medical illustrations, 2D videos, or textual descriptions cannot replicate the actual perception of patients suffering from retinal diseases. As a result, medical students, healthcare professionals, and the general public often struggle to empathize with or fully comprehend the lived experience of visual loss.

This lack of immersive educational tools leads to an empathy gap that affects both clinical communication and public awareness. Furthermore, most available systems that attempt to visualize eye conditions are either static, lack interactivity, or require expensive hardware. There is also limited integration between immersive simulations and intelligent educational assistance, which restricts their usability as comprehensive learning tools.

The problem addressed by this project is the absence of an accessible, web-based, and interactive simulation platform that can accurately reproduce visual distortions caused by retinal diseases, while simultaneously offering AI-driven medical guidance. By combining Virtual Reality (VR) technology with AI conversational support, this project aims to bridge the gap between scientific understanding and sensory experience, promoting empathy, awareness, and preventive health education.

Chapter 2

Background and Previous Work

Chapter 2: Background and Previous work

2.1 Background

2.1.1 Virtual Reality (VR):

Virtual Reality (VR) is a technology that immerses users in a computer-generated 3D environment that can simulate real or imagined scenarios. Through head-mounted displays (HMDs), motion sensors, and controllers, users can interact naturally with digital environments in a realistic way. Its main goal is to create immersive experiences that let users observe and respond as if they were physically present. VR is widely used in training, education, simulation, awareness, and entertainment. In healthcare, it supports empathy-based simulations and medical training by replicating real-world challenges and patient experiences to enhance understanding and care.

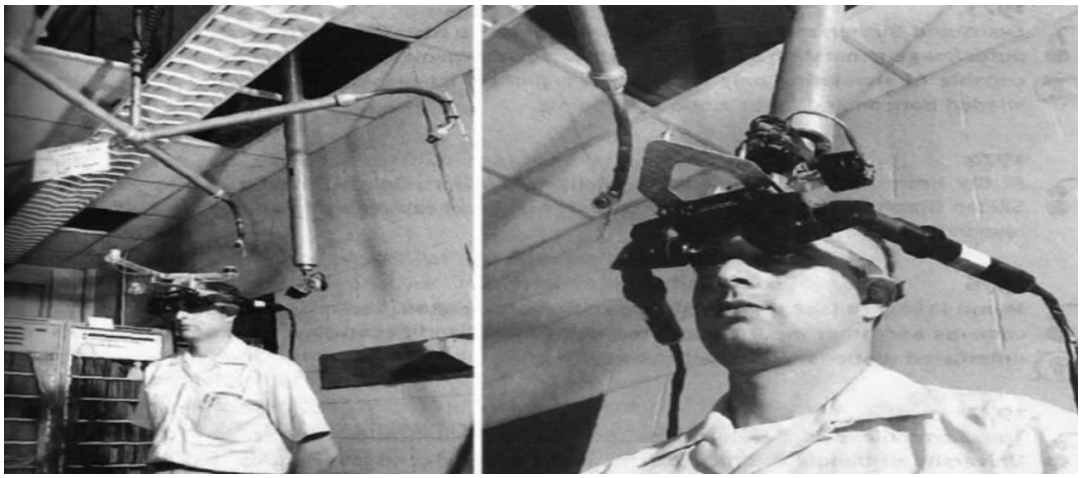


Figure 1: First HMD developed by Ivan Sutherland in 1968

2.1.1.1 Common Frameworks/Technologies:

- **Unity 3D:** Cross-platform engine used for building 3D and VR applications with interactive physics and lighting.
- **Unreal Engine:** Provides high-end, realistic rendering and simulation for VR projects.
- **WebXR / WebVR:** Browser-based APIs for running immersive VR experiences directly online.
- **OpenXR:** Standardized API that ensures compatibility across different VR platforms and devices.

- **A-Frame / Three.js / Babylon.js:** JavaScript-based frameworks used for lightweight, browser-compatible VR development.
- **Oculus SDK / SteamVR / Mixed Reality Toolkit:** Software kits for connecting applications with VR hardware.

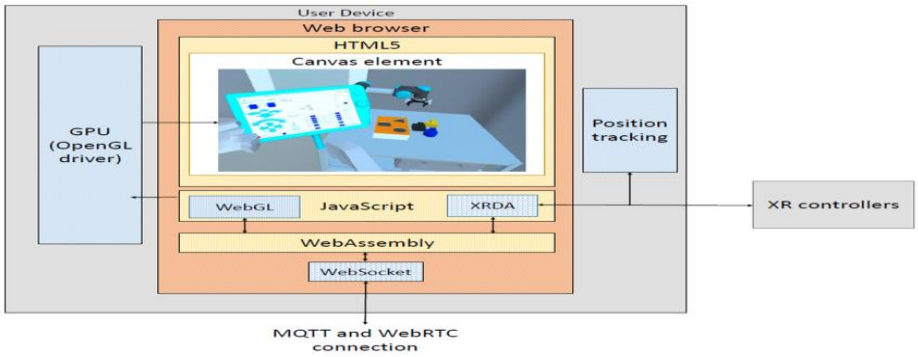


Figure 2: Generic web-based XR architecture

2.1.1.2 Types of VR Systems:

- **Fully Immersive VR:**
  The most advanced form, offering full visual, auditory, and motion immersion through HMDs and sensors (e.g., Oculus Quest, HTC Vive).
- **Semi-Immersive VR:**
  Uses large displays or projection systems for partial immersion (e.g., flight simulators, visualization labs).
- **Non-Immersive VR:**
  Displays 3D content on traditional screens, offering limited interaction (e.g., desktop simulators).

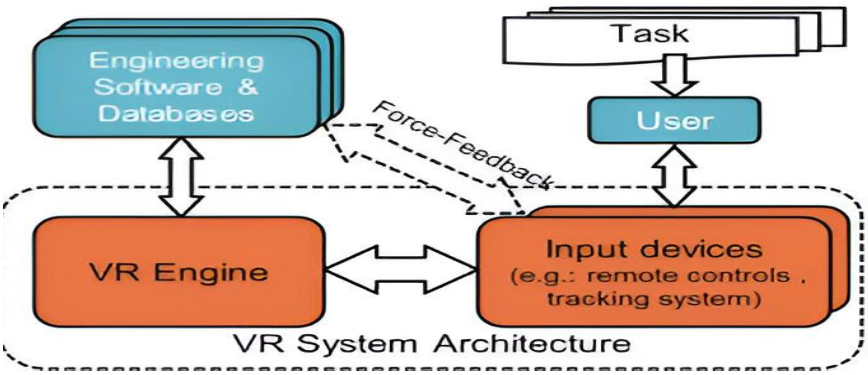


Figure 3: Classic VR system architecture

2.1.1.3 Types of VR Architectures:

Standalone VR Architecture

- **Definition:** A self-contained VR setup that integrates the processor, sensors, and display within the headset.
- **Purpose:** To provide wireless, mobile, and portable VR experiences without external hardware.
- **Common Frameworks/Technologies:** Meta Quest 2 / 3, Pico Neo, Snapdragon XR processors.
- **Use Cases:** Awareness training, educational simulations, and lightweight applications.

PC-Tethered VR Architecture

- **Definition:** A headset connected to a high-performance PC that handles rendering and computation.
- **Purpose:** To deliver high-quality graphics and low latency for professional VR simulations.
- **Common Frameworks/Technologies:** HTC Vive Pro, Valve Index, Oculus Rift, SteamVR, NVIDIA RTX GPUs.
- **Use Cases:** Industrial design, healthcare simulation, and research applications.

Mobile VR Architecture

- **Definition:** A smartphone-based VR approach where the phone acts as both the display and processor.
- **Purpose:** To make VR affordable and widely accessible for general users.
- **Common Frameworks/Technologies:** Google Cardboard, Samsung Gear VR, Daydream VR.
- **Use Cases:** Awareness campaigns and lightweight educational content.

Web-Based VR Architecture

- **Definition:** A browser-accessible VR model built using WebXR APIs, requiring no installation.

• **Purpose:** To maximize accessibility and cross-platform usability.
• **Common Frameworks/Technologies:** A-Frame, Three.js, Babylon.js, PlayCanvas.
• **Use Cases:** Web-delivered VR simulations, empathy training, and online exhibitions.

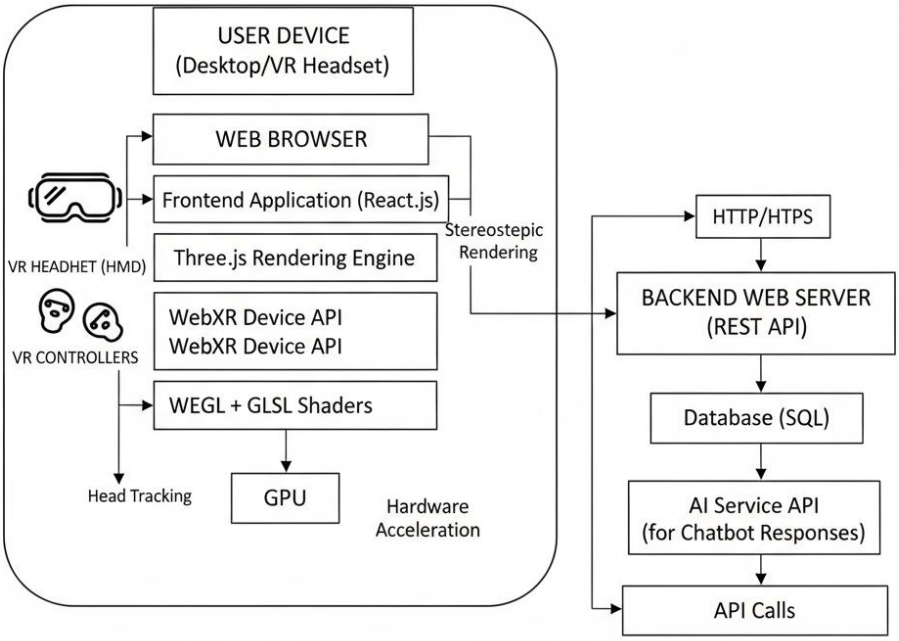


Figure 4: web-Based VR System Architecture of the Proposed Platform

Cloud VR Architecture

• **Definition:** A cloud-based setup where rendering happens remotely and is streamed to the headset.
• **Purpose:** To provide high-end VR performance on low-end or mobile devices.
• **Common Frameworks/Technologies:** NVIDIA CloudXR, AWS Wavelength, Google Cloud Streaming.
• **Use Cases:** Enterprise-level VR training, remote collaboration, and large-scale simulations.

2.1.1.4 Key Components of VR Systems:

- **Hardware:**
  Includes headsets (HMDs), controllers, sensors, tracking systems, and haptic devices that detect user movement and provide feedback.

- **Software:**
  Game engines, SDKs, and APIs that handle graphics, motion, and user interaction (e.g., Unity, Unreal, WebXR).
- **Interaction Systems:**
  Translate physical movement into digital actions using motion tracking (6DoF), gesture recognition, and eye tracking.
- **Rendering Systems:**
  Use graphics APIs like OpenGL, DirectX, and Vulkan to create realistic 3D visuals for both eyes.
- **Haptic Feedback:**
  Adds tactile sensations to enhance immersion using vibration or resistance (e.g., gloves, controllers).

2.1.1.5 Performance Optimization Techniques in VR:

- **Level of Detail (LOD):** Simplifies distant objects to maintain smooth rendering.
- **Frustum Culling:** Excludes non-visible objects to save processing power.
- **Dynamic Resolution:** Automatically adjusts rendering resolution based on system load.
- **Foveated Rendering:** Focuses high detail only where the user's eyes are looking.
- **Motion Smoothing:** Reduces latency and enhances comfort during motion.

2.1.2 Frontend Concepts:

The Frontend refers to client-side part of an application — the layer that users directly interact with through a web browser or mobile interface. It is responsible for displaying data, managing user interactions, and ensuring an intuitive and responsive user experience. A good frontend architecture focuses on performance, usability, and scalability by efficiently handling rendering and communication with the backend. Modern frontend development uses frameworks such as React.js, Angular, and Vue.js to build interactive, dynamic, and maintainable interfaces with real-time updates and seamless navigation.

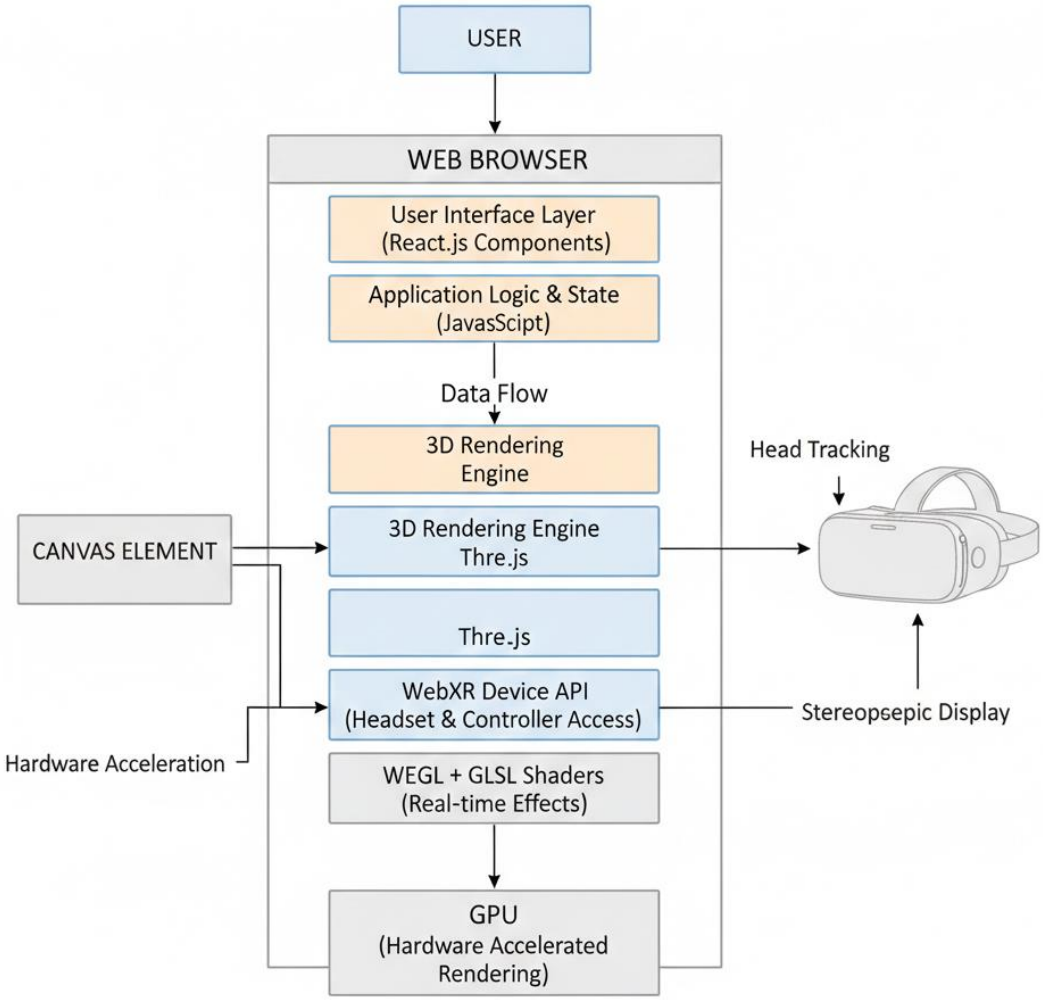


Figure 5: Frontend Runtime Architecture of the Web-Based VR System

2.1.2.1 Types of Frontend Architectures (Rendering Patterns):

- **Client-Side Rendering (CSR) / Single-Page Application (SPA):**
  - **Definition:** An architecture where the server sends a minimal HTML shell and a large JavaScript bundle to the browser. The JavaScript then takes over, dynamically rendering content, handling user interactions, and managing routing on the client-side.
  - **Purpose:** To create a fast, fluid user experience similar to a desktop application, eliminating full-page reloads and enabling complex, stateful interfaces.
  - **Common Frameworks/Technologies:**

    - **React.js:** A component-based JavaScript library for building user interfaces, often using create-react-app for setup
    - **Angular:** A comprehensive, TypeScript-based platform and framework for building large-scale, enterprise-level SPAs.
    - **Vue.js:** A progressive framework known for its component-based architecture and approachability, often used for SPAs.
    - **Svelte:** A compiler that converts components into highly efficient, imperative JavaScript code at build time, rather than running in the browser.

- **Server-Side Rendering (SSR):**
  - **Definition:** A traditional rendering pattern where the server generates the full, complete HTML for a page in response to each browser request. The browser receives a ready-to-display page
  - **Purpose:** To improve initial page load performance (especially Time to First Byte) and enhance Search Engine Optimization (SEO), as web crawlers receive complete content
  - **Common Frameworks/Technologies:**
    - **Next.js (React):** A "meta-framework" for React that provides SSR, routing, and other production-grade features by default**.**
    - **Nuxt.js (Vue):** The equivalent of Next.js for the Vue ecosystem, enabling server-side rendering.
    - **ASP.NET Core (Razor Pages / MVC):** Microsoft's backend framework, which includes powerful templating engines for rendering HTML on the server.
    - **Django (Python) / Ruby on Rails / Laravel (PHP):** Monolithic backend frameworks that use built-in templating engines (e.g., DTL, ERB, Blade) to render HTML on the server.

- **Static Site Generation (SSG):**
  - **Definition:** An approach where all website pages are pre-rendered into static HTML, CSS, and JavaScript files at *build time*. These static files are then deployed to a server or Content Delivery Network (CDN).
  - **Purpose:** To deliver maximum performance, security, and scalability at a low cost, as the server's only job is to return pre-built files. Ideal for sites where content doesn't change frequently (e.g., blogs, documentation, portfolios).
  - **Common Frameworks/Technologies:**

- **Hugo:** A high-performance static site generator written in Go, known for its incredibly fast build speeds.
- **Jekyll:** A popular SSG written in Ruby, famously used by GitHub Pages.
- **Eleventy(11ty):** A modern, flexible, JavaScript-based generator known for its simplicity.
- **Next.js / Gatsby:** React-based frameworks that also excel at SSG, often combining it with client-side code ("hydration") to create fast, interactive sites.

2.1.3 Backend Concepts:

The Backend refers to the server-side part of an application — the layer responsible for managing data, executing business logic, and handling operations that are hidden from the end user. It supports the frontend by processing requests, managing databases, authenticating users, and ensuring that data is delivered securely and efficiently. A well-structured backend architecture focuses on performance, scalability, and reliability, enabling smooth communication between the client and server through Application Programming Interfaces (APIs). These APIs act as a bridge between the frontend interface and backend logic, ensuring that user actions result in accurate and timely responses.

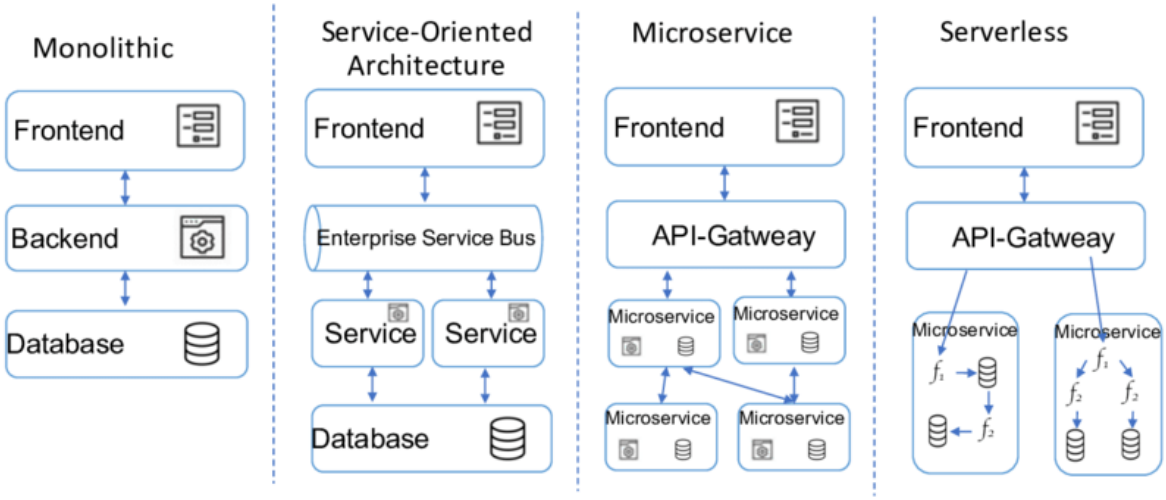


Figure 6: Comparison of Common Backend Architectural Styles

2.1.3.1 Types of Backend Architectures:

- **Monolithic Architecture:**
  - **Definition:** A traditional model where all components of the application (such as business logic, data access, and user authentication) are developed, coupled, and deployed as a single, indivisible unit.
  - **Purpose:** To simplify development and deployment for smaller applications by having all code in one codebase, sharing the same memory and resources.
  - **Common Frameworks/Technologies:**
    - **Ruby on Rails:** A framework that follows the model-view-controller (MVC) pattern, often used for rapid development
    - **Django (Python):** A high-level Python framework that encourages rapid development and clean, pragmatic design.
    - **Spring Boot (Java):** An extension of the Spring framework used to create stand-alone, production-grade applications**.**
    - **Laravel (PHP):** A web application framework with expressive, elegant syntax, often used for content management systems.
    - **ASP.NET MVC (C#) / ASP.NET Core MVC (C#):** A framework from Microsoft that uses the Model-View-Controller pattern to build a single, coupled application

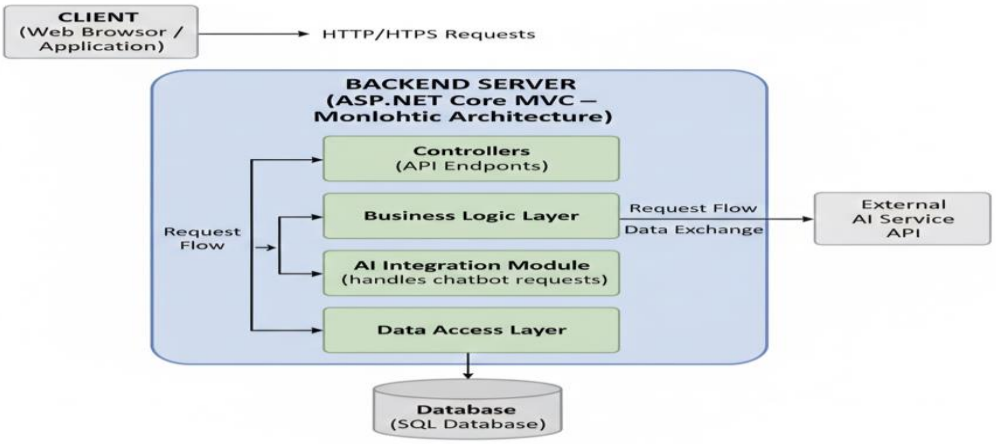


Figure 7: Monolithic Backend Architecture of the Proposed System

- **Microservices Architecture:**
    - **Definition:** An architectural style that structures an application as a collection of small, independently deployable services, each focused on a specific business capability.
    - **Purpose:** To improve scalability, flexibility, and fault isolation in large, complex applications by allowing teams to develop, deploy, and scale services independently.
    - **Common Frameworks/Technologies:**

1. **Node.js (Express.js/Fastify):** Used for building fast and lightweight APIs that serve as the communication layer for services.
2. **Docker:** A containerization platform used to package and deploy individual microservices consistently.
3. **Kubernetes:** An orchestration system for automating the deployment, scaling, and management of containerized services.
4. **REST APIs / gRPC:** Communication protocols used for services to interact with each other.
5. **Kafka / RabbitMQ:** Message brokers used for asynchronous communication between services.

- **Serverless Architecture (FaaS):**
    - **Definition:** A cloud-native model where the cloud provider manages the server infrastructure, and the application code is executed in stateless compute containers that are event-triggered and ephemeral (Function-as-a-Service).
    - **Purpose:** To reduce operational overhead and cost by eliminating the need to manage servers and by paying only for the compute time consumed.
    - **Common Platforms/Technologies:**
        - **AWS Lambda:** A serverless compute service from Amazon Web Services that runs code in response to events.

- **Google Cloud Functions:** Google's event-driven serverless compute platform.
- **Azure Functions:** A serverless solution from Microsoft for running event-triggered code.
- **API Gateway:** A service (like AWS API Gateway) that acts as the front door for applications to access backend functions via HTTP requests**.**

2.1.4 Artificial Intelligence (AI):

Key Components:

- **Definition:** An Application Programming Interface (API) provided by OpenAI that allows developers to send requests and receive responses from powerful, pre-trained large language models (LLMs) like GPT-4 or GPT-3.5-turbo
- **Purpose:** To serve as the "brain" of the chatbot. It handles the core Natural Language Processing (NLP) tasks, including understanding the user's query (intent) and generating a relevant, human-like response.
- **Common Elements:**
    - **API Key:** A private security token used to authenticate requests and bill for usage.
    - **Endpoint:** The Specific URL (e.g., api. openai/v1/chat/ completions) where the backend sends requests.
    - **Prompt Engineering:** The technique of carefully crafting the input (prompt), which often includes system instructions, the conversation history, and the user's new message, to guide the AI's response

2.2 Previous work

A review of existing literature and projects reveals that the use of Virtual Reality (VR) to simulate visual impairments is a well-established field, primarily focused on enhancing empathy and education. However, key gaps remain in accessibility and integrated learning, which this project aims to address.

Previous work can be broadly categorized into native VR applications for clinical education, public awareness tools, and specialized high-fidelity research platforms.

- **Academic and Usability Studies:** A significant body of research has focused on developing and validating native VR applications for empathy training in medical settings. For instance, a study published in *PLOS Digital Health* and featured in *ResearchGate* and *EyeNews* details the development of a VR-based application to simulate conditions like age-related macular degeneration (AMD), glaucoma, and cataracts [2, 3, 4]. This application was tested for its usability and effectiveness among medical students and staff, demonstrating that such simulations are valuable and well-received educational tools for enhancing empathy [4]. These studies primarily focus on standalone or PC-tethered VR headsets, requiring dedicated software installation.
- **Public Awareness and Commercial Tools:** Several organizations have created VR experiences to raise public awareness. The National Eye Institute (NEI), for example, developed the "See What I See" VR experience [6]. This application was designed for public outreach and education, simulating various eye diseases on mobile and standalone VR platforms to help the public and patients' families understand the sensory experience of visual impairment. Similarly, "EmpathEyes" exists as a dedicated software solution for healthcare professionals, caregivers, and educators to experience simulations of conditions like glaucoma and diabetic retinopathy, again delivered as a specific application for VR hardware [5].
- **High-Fidelity Research Platforms:** More advanced projects, such as the "XREye" by VRVis, focus on creating medically-plausible, real-time VR and AR simulations of visual impairments [1]. This project is aimed at a specialized audience, including ophthalmologists, caregivers, and even architects, to help them understand the functional impact of eye diseases. This represents a high-fidelity, expert-level tool rather than a widely accessible educational platform.

2.2.1 Academic and Clinical Research Platforms

XREye (VRVis)

A high-fidelity VR/AR system for simulating visual impairments.

Strengths:

- Medically accurate simulations
- Real-time distortion rendering

Limitations:

- Requires advanced hardware
- Not web-based
- No AI interactivity

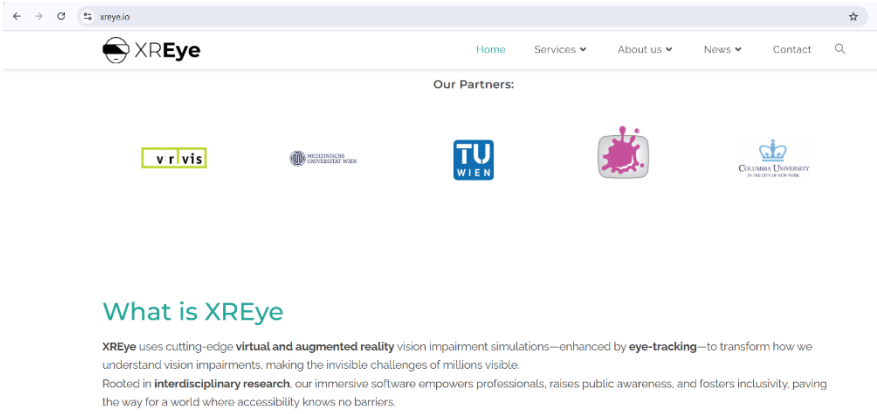


Figure 8: XREye Visual Impairment Simulation

2.2.2 Public Awareness VR Tools

NEI "See What I See" (National Eye Institute)

A VR experience primarily for public education. The experience is designed to be user-friendly, requiring minimal setup, and works on common standalone VR devices. However, the distortions are simplified approximations rather than fully medically accurate shader models.

Strengths:

- Reliable medical source
- Easy to use
- Useful for general awareness and education

Limitations:

- Simple distortions
- Not interactive
- No disease progression control
- Not browser-based

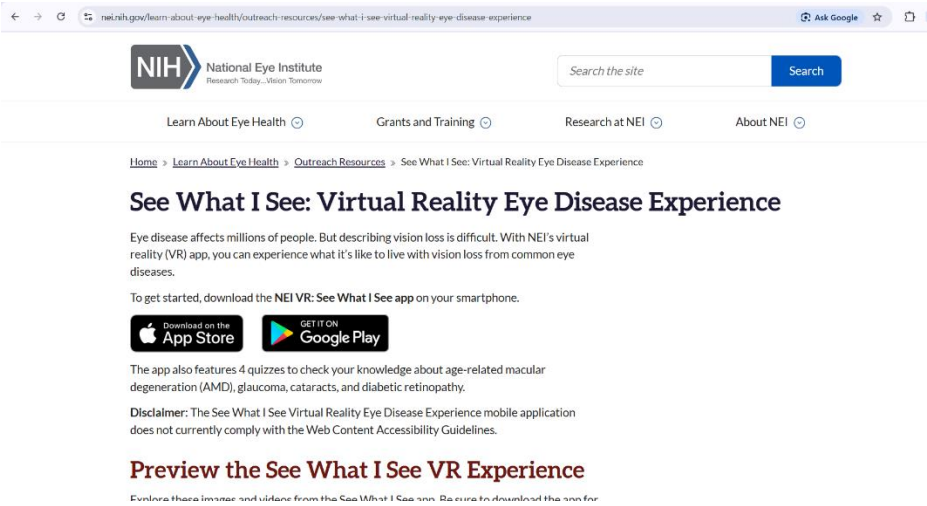


Figure 9: NEI "See What I See" (National Eye Institute)

EmpathEyes Vision Simulator

Provides basic immersive distortions for vision conditions.

Strengths:

- Good for training
- Simple interaction

Limitations:

- Limited realism
- No dynamic progression of diseases
- No progression or AI support

Figure 10: EmpathEyes Vision Simulator

2.2.3 Light Web-Based Tools

VisionSim (Web-Based Distortion Simulator)

VisionSim is a lightweight 2D web-based tool that applies visual distortion filters on top of static images to simulate different eye conditions such as cataracts, macular degeneration, and diabetic retinopathy. The system typically uses simple shader-like overlays or central scotoma filters to approximate how patients perceive their surroundings.

Strengths:

- Accessible through any standard web browser.
- No installation or specialized hardware required.
- Useful for basic public awareness and quick demonstrations.

Limitations:

- Only modifies *2D images*, not fully immersive 3D scenes.
- Simulations are static and not affected by user movement or depth.
- No real-time interaction, progression control, or environmental context.

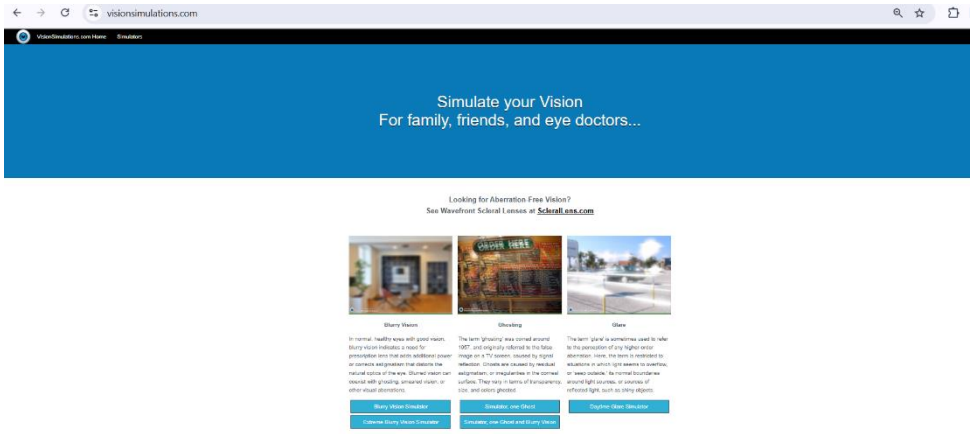


Figure 11: Example of a Light Web-Based Distortion Tool (VisionSim)

2.2.4 Identified Gaps in Previous Work

While the reviewed projects successfully demonstrate VR's power for empathy and education, they highlight a significant gap that "Through the Eye" directly addresses. The vast majority of existing solutions are native applications that require specific VR hardware (like Oculus Quest or PC-tethered systems) and software installation [4, 5, 6]. This creates a barrier to accessibility for the general public and for educational institutions without a large budget for hardware.

Furthermore, these existing systems are primarily passive visual simulators. They effectively show *what* a patient sees but lack an interactive educational component. Users are left to experience the visual loss without asking *why* it's happening or *what* can be done about it.

This project fills this critical gap by:

- **Prioritizing Accessibility:** Utilizing WebXR and Three.js, our solution is browser-based, eliminating the need for app installation and making it accessible on a wider range of devices, from desktop browsers to standalone headsets.
- **Integrating AI-Powered Guidance:** Unlike previous work, our project introduces an AI-powered chatbot. This key feature transforms the experience from a passive simulation into an interactive learning tool.

Chapter 3

Planning and Analysis

Chapter 3: Planning and Analysis

3.1 Planning

3.1.1 Feasibility study and estimated cost

- **Technical Feasibility**

The technical feasibility of the project evaluates whether the system can be developed and implemented using the available hardware, software, and technical skills.

The proposed system, “Through the Eye: VR Simulation of Retinal Diseases with AI Chatbot,” is technically feasible because it relies on widely available and well-supported technologies. The development requires a VR headset such as Meta Quest or any WebXR-compatible device, along with a VR-ready computer equipped with a capable GPU for testing and performance optimization.

On the software side, the system utilizes Three.js, WebGL, and GLSL for 3D rendering and real-time visual simulations, while Blender is used for creating and optimizing 3D models. The frontend is built using React.js and Tailwind CSS, ensuring responsiveness and interactivity. The integration of the OpenAI API (GPT) provides intelligent chatbot responses that enhance user awareness and education.

All required software tools are open-source or have publicly available APIs, making development cost-effective and accessible. The system runs on modern WebXR-supported browsers such as Google Chrome and Microsoft Edge, which ensures cross-platform compatibility and ease of use.

Given the team’s existing skills in 3D modeling, shader programming, and AI integration, and the availability of both hardware and software resources, the project is considered technically achievable and sustainable within the planned development period.

- **Operational Feasibility**

The operational feasibility determines whether the proposed system can be successfully used and maintained in its intended environment.
It evaluates how well the system fits into the users' needs, organizational workflow, and overall usability.

The proposed "*Through the Eye*" system is operationally feasible as it provides an easy-to-use, educational, and interactive experience
for both medical and non-medical users. The system allows users to explore how retinal diseases affect human vision through an immersive VR environment, while also interacting with an AI-powered chatbot that offers medical awareness and preventive information.

- Ease of Use
  The system features a simple and intuitive user interface, allowing users to select a disease, adjust severity, and navigate the VR environment easily.
  Both VR controllers and standard input devices (keyboard and mouse) can be used, making it accessible for users with or without VR headsets.
  The AI chatbot is text-based, requiring no prior technical knowledge to interact with.

- Accessibility and Compatibility
  The application runs directly in modern web browsers that support WebXR, such as Google Chrome and Microsoft Edge, without requiring installation.
  It can be deployed in various environments — including universities, hospitals, or educational exhibitions — with minimal setup.
  This makes the system accessible to a wide audience using standard hardware and internet access.

- User Acceptance
  The project offers high user engagement because of its immersive design and interactive educational content.

Medical students, doctors, and general users can benefit from understanding retinal diseases visually rather than through traditional text-based materials.
The AI chatbot further enhances acceptance by providing personalized and clear responses during the simulation.

- Maintainability and Support
  The system is designed with a modular structure, allowing easy updates for adding new diseases, improving shaders, or expanding chatbot content.
  Maintenance tasks such as content updates or performance optimization can be performed without major rework.
  Documentation and version control (via GitHub) ensure smooth collaboration among developers.
- Training and Operation
  Minimal training is required for operators or instructors to use the system.
  A short tutorial or user guide can easily help users understand navigation and controls.
  The simplicity of the user interface supports both self-directed learning and guided demonstrations.

- **Estimated Cost**
  The estimated cost of developing the "*Through the Eye*" system is moderate, as most tools used are open-source or freely available.
  The main expenses are related to hardware, API usage, and testing. The following table outlines the approximate cost distribution:

Total Estimated Initial Cost: ≈ 1,300 – 1,900 USD

Table 1: estimated cost

Category	Item	Estimated Cost (USD)	Notes
Hardware	VR Headset (Meta Quest or equivalent)	50$	One-time rental for testing and user experience
	VR-ready Computer (high GPU performance)	800$ – 1,000$	Used for development and 3D rendering
Software	Three.js, WebGL, React.js, Tailwind CSS	Free	Open-source and cost-free tools
API Services	OpenAI GPT API Subscription	0$ - 50$	Based on usage for chatbot interaction
Development Tools	GitHub (version control), Blender, Visual Studio Code, VS Code	Free	Freely available online
Testing & Deployment	Web Hosting / Domain for prototype	50$ – 100$	For public access and demo hosting
Miscellaneous	Backup storage, minor tools	5-15	Optional or maintenance costs

3.1.2 Gantt Chart

Project Plan

Project plan is a calendar that links the tasks to be done with the resources that will do them. The project plan defines the work that will be done on the project and who will do it.

	A	B	C	D
1	Task	Start Date	End Date	Duration
2	Feasibility Study	10/15/2025	10/24/2025	10
3	Gather Information	10/25/2025	11/13/2025	20
4	Design ERD	11/14/2025	11/23/2025	10
5	Design UML Diagrams	11/24/2025	12/23/2025	30
6	Design UI/UX	12/24/2025	12/28/2025	5
7	Implement Frontend	12/29/2026	1/12/2026	15
8	Implement Backend	1/13/2026	1/27/2026	15
9	Implement VR Environment & Filters	1/28/2026	2/28/2026	32
10	Software Testing	1/3/2026	3/16/2026	16

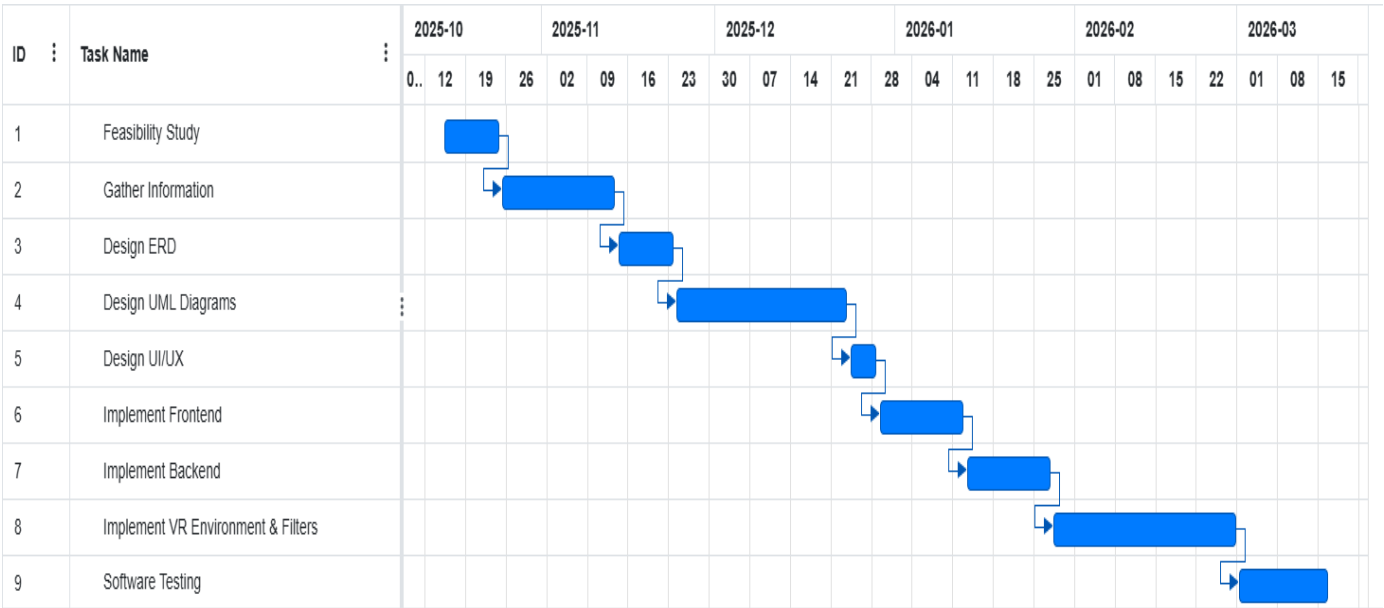


Figure 12: Gantt Chart

3.2 Analysis and Limitations of Existing System

3.2.1 Analysis of Existing Systems

- **XREye (VRVis)**

XREye is one of the most advanced retinal disease simulators, offering medically accurate visual distortions. However, it relies on standalone, high-performance setups and is not accessible to general users.

Strengths

- High-fidelity medically accurate simulations
- Realistic 3D environments
- Strong educational value for specialized institutions

Weaknesses

- Requires installation and powerful hardware
- Not browser-based, limiting accessibility
- No interactive educational component (no chatbot)
- Not suitable for public or large-scale awareness environments

- **EmpathEyes Vision Simulator**

A simple vision-loss awareness tool that shows approximations of visual conditions.

Strengths

- Easy to use
- Provides visual demonstrations of basic conditions

Weaknesses

- Not a full 3D simulation
- Limited visual accuracy and customization
- No dynamic disease progression
- No interactive learning or AI support

- **NEI "See What I See"**

A government awareness tool designed for simple public understanding, not for advanced simulation.

Strengths

- Trusted medical source
- Good for basic awareness

Weaknesses

- Very simple visual representation
- Not a 3D environment
- No progression control or interactive features
- No chatbot or intelligent educational support
- Not fully web-based
-
- **Visual Impairment Experience Simulator (Web App)**

A lightweight web-based tool that simulates visual distortions on 2D images.

Strengths

- Web-based and easy to access
- Simple and fast to use

Weaknesses

- Only 2D simulation
- No movement or environment interaction
- Limited accuracy and realism
- No AI fields or intelligent assistance
- Not suitable for medical or educational depth

3.2.2 Limitations of Existing Systems

- **High Hardware Requirements**
  - Some simulators, like XREye, require powerful standalone hardware and installation, limiting accessibility for general users.
- **Limited Web Accessibility**
  - Most systems are not fully web-based, making them less accessible for public awareness or large-scale deployment.
- **Lack of Full 3D Simulation**
  - Tools like EmpathEyes and Visual Impairment Experience Simulator offer only 2D simulations or limited 3D environments, reducing realism.
- **No Interactive or AI Support**
  - Existing systems lack interactive learning features such as chatbots, AI-assisted guidance, or progression control for diseases.
- **Limited Educational Depth**
  - Many tools are suitable only for basic awareness, not for detailed medical or educational purposes.
- **No Dynamic Disease Progression**
  - Current simulators do not provide the ability to simulate how vision impairment progresses over time.
- **Restricted Customization**
  - Users cannot adjust visual conditions or environments extensively in most existing tools.

3.3 Need for New System

Based on the analysis of existing systems and their limitations, there is a clear need for a new vision simulation system that addresses the following requirements:

- **Web-Based Accessibility**
  a. The new system should be fully web-based, allowing easy access for users without the need for powerful hardware or installation.
- **Realistic 3D Simulation**

a. It should provide realistic 3D environments that better represent visual impairments, enhancing understanding and immersion.

- **Interactive Learning and AI Support**
  a. Incorporating AI features such as chatbots and intelligent guidance will allow interactive educational experiences and personalized learning.
- **Dynamic Disease Progression**
  a. Users should be able to observe how visual impairments progress over time, offering deeper insight into retinal diseases.
- **Customization and Flexibility**
  a. The system should allow users to adjust visual conditions and environments according to different scenarios for better awareness and training.
- **Educational and Awareness Applications**
  a. It should be suitable for both medical education and public awareness campaigns, bridging the gap between specialized institutions and general users.
- **Lightweight and Accessible Design**
  a. Optimized for performance so it can run smoothly on standard devices without sacrificing realism or interactivity.

3.4 Analysis of new system

3.4.1 User Requirements

Table 2: User Requirements

Code	Requirement Statement	Priority	Notes
UR01	User Registration and Authentication: Users must be able to create an account and sign in using email and password.	Must	Supports role-based access (User, Admin).

(continued from previous page)

Code	Requirement Statement	Priority	Notes
UR02	View Disease Educational Information: Users must be able to browse and view detailed educational content for each retinal disease.	Must	Includes causes, symptoms, progression, and prevention.
UR03	Interact with AI Chatbot: Users must be able to ask questions and receive AI-generated responses about retinal diseases.	Must	-
UR04	Start and Control VR Simulation: Users should be able to launch a VR simulation, select diseases, and adjust severity levels in real time.	Must	-
UR05	Navigate Virtual Environment: Users should be able to move and look around within the 3D VR environment.	Should	Supports keyboard/mouse and VR controllers.
UR06	Account Management: Users should be able to update profile settings and view simulation history.	Should	-
UR07	Admin Dashboard Access: Administrators should have access to a dedicated panel for managing content and users.	Must	-

3.4.2 System Requirements

Table 3: System Requirements

Code	Requirement Statement	Priority	Notes
Hardware Requirements			
SR01	VR Headset Compatibility: The system must support WebXR-compatible headsets (e.g., Meta Quest) and desktop browsers.	Must	Required for development only; not for end-users.
SR02	Development and testing should be performed on a VR-ready computer with a dedicated GPU (e.g., NVIDIA RTX series or equivalent) to ensure real-time simulation performance and shader development.	Should	-
SR03	Input Device Support: The system must support standard keyboard and mouse for desktop navigation, and VR controllers for immersive interaction within the virtual environment.	Must	Dual-mode input for accessibility.
SR04	Network Connectivity: A high-speed internet connection (Wi-Fi or Ethernet) is required for seamless loading of high-resolution textures, 3D assets, and low-latency AI chatbot interactions.	Must	-
Software Requirements			
SR05	Web Browser Support: The system must run on latest versions of Chrome, Edge, and other WebXR-supported browsers.	Must	-
SR06	Backend Framework: The system must use ASP.NET Core MVC for business logic and API management.	Must	SQL Server database integration.

(continued from previous page)

Code	Requirement Statement	Priority	Notes
SR07	Frontend Framework: The system must use React.js for building a responsive and interactive user interface.	Must	Tailwind CSS for styling.
SR08	3D Rendering Engine: The system must use Three.js and WebGL for real-time 3D rendering and shader effects.	Must	GLSL for custom post-processing shaders.
SR09	AI Integration: The system must integrate OpenAI API for chatbot functionality via secure backend endpoints.	Must	-
SR10	3D Asset: The system should include optimized 3D model and textures for VR environment.	Should	Low-poly model for performance.
SR11	Integrated Development Environment: System development should utilize a modern IDE such as Visual Studio Code / Visual Studio with appropriate extensions for React, C#, and Three.js development.	Should	Facilitates team collaboration and code maintenance.
SR12	Database Management: The system must use SQL Server to store user accounts, and disease content.	Must	-

3.4.3 Domain Requirements

Table 4: Domain Requirements

Code	Requirement Statement	Priority	Notes
DR01	Medical Accuracy: All visual simulations must be based on medically verified ophthalmological data.	Must	References: AAO, NEI, NIH.
DR02	Realistic Visual Effects: Shader-based distortions must accurately mimic real retinal disease symptoms.	Must	-
DR03	Severity Adjustment Consistency: Disease severity levels must correspond to actual medical progression stages.	Must	Slider-based real-time adjustment.
DR04	Educational Integrity: Chatbot and content must provide awareness only. No diagnosis or treatment advice.	Must	Clear disclaimers required.
DR05	Data Privacy Compliance: All AI interactions and system operations must comply with data privacy standards. No personal data shall be stored.	Must	Supports keyboard/mouse and VR controllers.
DR06	Terminology Standards: Medical terminology and descriptions must align with professional healthcare standards.	Must	Based on trusted medical sources.
DR07	Cross-Platform Accessibility: System must be accessible on desktop, mobile, and VR with consistent experience.	Must	WebXR for VR, responsive design for non-VR.

3.4.4 Functional Requirements

3.4.4.1 Functional Requirements for regular users

Table 5: Regular User Functional Requirements

Code	Requirement Statement	Priority	Traces To
User Authentication			
FR01	Allow sign-up, login, and logout with email/password.	Must	UR01
View Disease List & Educational Information			
FR02	View Disease List: Display all available retinal diseases with brief descriptions.	Must	UR02
FR03	View Disease Details: Show detailed educational information for selected disease.	Must	UR02
Interact with AI Chatbot			
FR04	Send user queries to OpenAI API and display responses in chat interface.	Must	UR03, SR09
Start, Navigate, and Control VR Simulation			
FR05	Start VR Simulation: Load 3D environment and apply selected disease's visual shaders.	Must	UR04, SR08
FR06	Adjust Disease Severity: Real-time slider control to modify shader parameters (e.g., blur, blind spots)	Must	UR04
FR07	Navigate in VR: Support movement and camera control via VR controllers or keyboard/mouse.	Should	UR05

3.4.4.2 Functional Requirements for Admin

Table 6: Admin Functional Requirements

Code	Requirement Statement	Priority	Traces To
User Authentication			
FR08	Admin Login: Secure admin access to authorized credentials.	Must	UR07
Manage User Accounts			
FR09	Manage user accounts, including viewing, updating, and removing user profiles as needed.	Must	UR07
Manage Disease Content			
FR10	Add, update, or remove retinal disease content, including disease information, shader configurations, and severity level parameters.	Must	UR07

3.4.5 Non-Functional Requirements

Table 7: Non-Functional Requirements

Code	Requirement Statement	Priority	Target
NFR01	VR Performance: The VR simulation must maintain a stable 60-72 FPS across all devices to ensure user comfort and prevent motion sickness during extended sessions.	Must	≥60 FPS
NFR02	AI Response Latency: The chatbot should respond within an acceptable time to maintain conversational flow and real-time educational support.	Should	≤7 sec

(continued from previous page)

Code	Requirement Statement	Priority	Target
NFR03	Visual Accuracy: All disease simulations must be medically accurate and validated against trusted ophthalmological sources to ensure educational integrity.	Must	Clinically reviewed
NFR04	Usability: The interface must be intuitive and easy to navigate for both medical professionals and general users, requiring minimal training.	Must	User testing feedback
NFR05	Reliability: The system must operate without crashes or significant performance degradation during typical VR simulation sessions.	Must	99% uptime
NFR06	Scalability: The architecture should content modules without major system changes.	Should	Modular architecture
NFR07	Accessibility: Support keyboard navigation and screen readers for non-VR mode.	Must	WCAG 2.1 Level A
NFR08	Maintainability: The codebase must be well-documented, modular, and follow coding standards for ease of future updates and collaboration.	Must	PEP 8 / ESLint standards

3.5 Advantages of the New System

The proposed system, "*Through the Eye*," offers several distinct advantages over traditional educational methods and existing native VR applications. By leveraging WebXR technology and Artificial Intelligence, the system addresses the limitations

of accessibility, interactivity, and cost found in previous solutions. The key advantages are as follows:

- **Enhanced Accessibility (Browser-Based Deployment)** Unlike traditional VR applications that require downloading large files, installing specific software, or using high-end tethered hardware, this system is entirely web-based.
  - **Zero Installation:** Users can access the simulation directly through a URL without installing third-party software.
  - **Cross-Platform Compatibility:** The system leverages WebXR, making it compatible with a wide range of devices, from dedicated VR headsets (like Meta Quest) to standard desktop browsers and mobile phones.
- **Integration of Intelligent Interactive Learning** Most existing visual simulators are passive; users observe the condition but cannot interact with it intellectually.
  - **AI Chatbot Support:** The integration of the OpenAI API transforms the system from a simple viewer into an active learning tool. Users can ask questions about symptoms, prevention, and treatment in real-time while experiencing the simulation.
  - **Personalized Guidance:** The AI adapts to the user's queries, providing immediate, context-aware medical information that static textbooks cannot offer.
- **Dynamic and Adjustable Disease Simulation** Textbooks and videos provide a static "snapshot" of a disease. The new system allows for dynamic manipulation of the visual experience.
  - **Real-Time Severity Adjustment:** Users can use sliders to adjust the severity of the disease (e.g., increasing the blur radius or the size of a blind spot). This helps users understand the *progression* of a disease, not just the end state.
  - **Immersive Empathy:** By simulating the functional burden of vision loss in a 3D environment, the system creates a deeper emotional and cognitive connection (empathy) than 2D illustrations.

- **Cost-Effectiveness** The system is designed to be economically viable for educational institutions and developing regions.
  - **Low Hardware Dependency:** It does not strictly require expensive, PC-tethered VR setups. It can run on standalone headsets or even standard computers.
  - **Open-Source Technologies:** The development relies on cost-free, open-source libraries (Three.js, React.js), reducing licensing costs compared to proprietary medical software.
- **Centralized Management and Scalability**
  - **Ease of Updates:** Since the application is hosted on the web, updates to the shaders, medical content, or AI prompts are instantly available to all users without requiring them to update their local software.
  - **Scalable Architecture:** The separation of the frontend (React) and backend (ASP.NET Core) allows for easy addition of new disease modules or features in the future without disrupting the existing system.

3.6 User Characteristics

Understanding the characteristics of the primary users of the system helps in designing an immersive, educational, and intuitive experience. The "Through the Eye" system accommodates two main user groups: Users and Administrators.

1. Users

Demographics:

Broad age range between 18–60 years old.

Includes general public individuals, medical students, and healthcare professionals interested in visual awareness or clinical understanding.

Users may vary in their technical background, ranging from highly tech-savvy individuals to those with minimal experience in VR tools.

Behavioral Traits:

Users are motivated to understand how retinal diseases affect vision through realistic, immersive simulation.

Prefer simple, intuitive interfaces that allow smooth navigation inside the VR environment.

Appreciate clear, educational explanations and real-time interaction with the AI chatbot to deepen their understanding.

Expectations:

Realistic and medically accurate visual simulations of different retinal diseases.

Easy-to-use controls for navigating the VR environment and adjusting disease severity.

Immediate access to educational content and AI-powered guidance throughout the simulation.

Ability to learn at their own pace without requiring advanced technical skills.

2. Administrators

Role:

Oversee the system's content, manage diseases, update educational material, and ensure smooth functionality.

Responsible for managing user accounts, monitoring system performance, and maintaining data integrity.

Expectations:

Access to a clear and user-friendly admin dashboard.

Tools to add new retinal diseases, update shaders or VR scenes, and modify chatbot content.

Ability to monitor system activity and ensure accuracy, stability, and reliability.

Chapter 4

Design

Chapter 4: Design

4.1 Design and Implementation Constraints

The design and implementation of the "*Through the Eye*" system is subject to a set of constraints that influence architectural decisions, technology selection, system performance, and overall development feasibility. These constraints arise from the nature of VR technology, web-based deployment, AI usage, medical accuracy requirements, and platform limitations.

4.1.1 Technological Constraints

WebXR Platform Limitations

- The system must operate entirely through WebXR, which has varying levels of support across browsers (Chrome, Edge support it; Safari and Firefox have limited or partial support).
- Advanced VR features available in engines like Unity or Unreal (e.g., baked lighting, high-end physics) cannot be used.
- Shader complexity must remain optimized to avoid performance drops.

Browser-Based Rendering Constraints

- WebGL/Three.js rendering is less powerful than native OpenGL or Vulkan engines.
- Real-time shader effects must remain lightweight to maintain VR comfort (90 FPS recommended for ideal VR experience).
- Complex 3D models or high-polygon environments may cause lag or nausea.

Hardware Restrictions

- The system must run on consumer-level VR headsets (e.g., Meta Quest) and standard PCs.
- Cannot rely on high-end GPUs typically used for clinical VR research (e.g., RTX 4090).
- Users with normal laptops (integrated GPU) should still be able to access the system in non-VR mode.

4.1.2 API and AI Constraints

OpenAI API Dependency

- Chatbot functionality requires stable internet; offline usage is not possible.
- API call limits and token restrictions may affect response length and interaction speed.
- API usage costs can influence long testing sessions or deployment scale.

Security Constraints

- The OpenAI API key must not be exposed in the frontend.
- API communication must be routed through a secure backend endpoint.

AI Response Constraints

- The chatbot must provide *awareness and educational information only*.
- It cannot generate medical diagnoses, prescriptions, or clinical decisions.

4.1.3 Medical Accuracy Constraints

Realistic Shader Requirements

- Visual distortions (blur, blind spots, tunnel vision, glare) must reflect ophthalmological literature.
- Creative or exaggerated distortions cannot be used unless medically validated.

Ethical Constraints

- The system must prevent misleading or exaggerated medical information.
- The simulation must remain educational, not diagnostic.

Data Constraints

- No patient-specific medical data may be collected or stored.
- All disease descriptions and visuals must follow verified medical sources.

4.1.4 Performance Constraints

VR Performance Requirements

- VR rendering must maintain high frame rates, ideally 72–90 FPS for comfort.
- Shader calculations must be optimized to avoid latency spikes that cause motion sickness.

Network Performance Constraints

- AI interaction depends on internet connectivity; poor networks may cause delayed responses.
- Multimedia content (3D models, textures) must be optimized for fast loading in browsers.

Device Compatibility

- Must run efficiently on standalone VR devices (Meta Quest) without overheating or throttling.
- Desktop fallback mode must operate smoothly without VR hardware.

4.1.5 Platform & Deployment Constraints

Cross-Browser Compatibility

- Must support at least:
  - Google Chrome
  - Microsoft Edge
- Safari and Firefox WebXR limitations restrict some features.

Backend Deployment Constraints

- ASP.NET Core backend must be hosted on compatible servers.
- SQL Server support required for database hosting (limits some free hosting providers).

Hosting Constraints

- WebXR content often requires HTTPS for full functionality.

- Large 3D assets must be served from optimized hosting/CDNs to maintain speed.

4.1.6 Development Constraints

Time Constraints

- The project must be completed within the academic timeline.
- Complex features like multiple disease shaders must be scoped appropriately.

Tool Limitations

- Multi-platform VR testing requires access to VR hardware during development.

4.1.7 Ethical and Legal Constraints

Ethical Use of AI

- AI cannot provide diagnosis or medical decision-making.
- The chatbot must present information responsibly to avoid misinformation.

Intellectual Property Constraints

- All medical descriptions must be original or properly cited.

User Privacy & Data Protection

- Only non-sensitive user data (email/username) may be stored.
- No medical, biometric, or personal health information can be collected

4.2 Assumptions and Dependencies

4.2.1 Assumptions

The following assumptions are considered true throughout the development, deployment, and use of the system:

- **Internet Connectivity**

It is assumed that users have access to a stable, high-speed internet connection**.** This is required for:

- Loading WebXR scenes and 3D assets
- Communicating with the backend
- Sending requests to the OpenAI API for chatbot responses
- Ensuring smooth VR performance

If the connection is unstable, AI responses may lag or fail.

- **Browser Compatibility**

It is assumed that users access the system through WebXR-compatible browsers, such as:

- Google Chrome
- Microsoft Edge
- Oculus Browser (Meta Quest)

Browsers that do not support WebXR may have limited or no VR functionality.

- **Supported Hardware Availability**

It is assumed that:

- Users have access to a **VR headset** (Meta Quest or equivalent), OR
- Users have a **standard PC/laptop** for non-VR mode

Additionally, VR controllers and sensors must function properly.

- **User Technical Literacy**

It is assumed that users:

- Understand basic navigation inside web interfaces
- Can follow on-screen instructions for entering VR mode
- Know how to use VR controllers or a mouse/keyboard
- Are familiar with reading educational content

This ensures smooth navigation and learning.

- **Medical Content and Shader Accuracy**

It is assumed that:

- All provided medical descriptions are correct and based on trusted sources
- Ophthalmological input (research, documentation) remains available and accessible
- Disease shader effects accurately reflect real visual distortions

This guarantees the educational value of the system.

- **AI Chatbot Reliability**

It is assumed that OpenAI API:

- Remains operational
- Returns responses within expected latency
- Maintains high availability and uptime

This ensures that users receive timely and accurate awareness responses.

- **Backend and Database Availability**

It is assumed that:

- The ASP.NET Core backend remains active and accessible
- SQL Server database runs without corruption
- Server hosting environment (cloud or local) remains operational and secure

This supports account management, content retrieval, and system functionality.

- **Disease Content Accuracy**

It is assumed that:

- Admins will upload medically accurate disease descriptions
- No misleading or fabricated information will be added
- Visual simulations and severity settings will follow validated medical references

This maintains scientific integrity and ethical standards.

- **Users Follow Ethical Use**

It is assumed that:

- Users will not misuse the system for clinical diagnosis
- Users understand that the system is for awareness, not treatment

4.2.2 Dependencies

The system relies on several software tools, hardware components, libraries, and external services. These dependencies must function properly for the system to operate.

- **Web Technologies**

The system depends on:

- **React.js** (frontend UI)
- **Three.js & WebGL** (3D rendering)
- **WebXR API** (VR support)
- **Tailwind CSS** (styling)

If any of these libraries break due to updates or deprecation, frontend functionality may be affected.

- **Backend Framework**

The backend depends on:

- **ASP.NET Core MVC** for routing, user management, and API handling
- **C# runtime** for logic execution
- **Entity Framework** for database communication

Any backend failure or misconfiguration disrupts the entire system.

- **Database**

The system depends on:

1. **SQL Server** for storing user accounts, disease content, and admin data
2. Entity relationships defined in the ERD
3. Data integrity constraints (foreign keys, validation rules)

A corrupted or unavailable database will affect account management and content loading.

- **AI API (OpenAI)**

The chatbot relies on:

- OpenAI API availability
- Valid API key
- Stable internet connection
- Proper prompt engineering configuration

If the service is down or the API key is invalid, chatbot features will not function.

- **VR Hardware and Sensors**

The system depends on:

- VR headset compatibility (Meta Quest recommended)
- Head tracking and controller input
- WebXR runtime provided by headset/browser vendors

Hardware malfunction will affect immersion and movement.

- **Hosting and Deployment**

The system depends on:

- A web server capable of hosting ASP.NET Core apps
- HTTPS support (mandatory for WebXR)

Without HTTPS, VR mode fails entirely.

- **Third-Party Libraries**

Additional dependencies include:

- **Blender** for model creation
- **GitHub** for version control

Version conflicts or dependency updates may affect stability.

- **Operating System Dependencies**

The system assumes:

- Windows for development
- VR headset OS (Meta Quest's Android-based system) supports WebXR
- Browsers running on these systems maintain compatibility

- **Documentation and Standards**

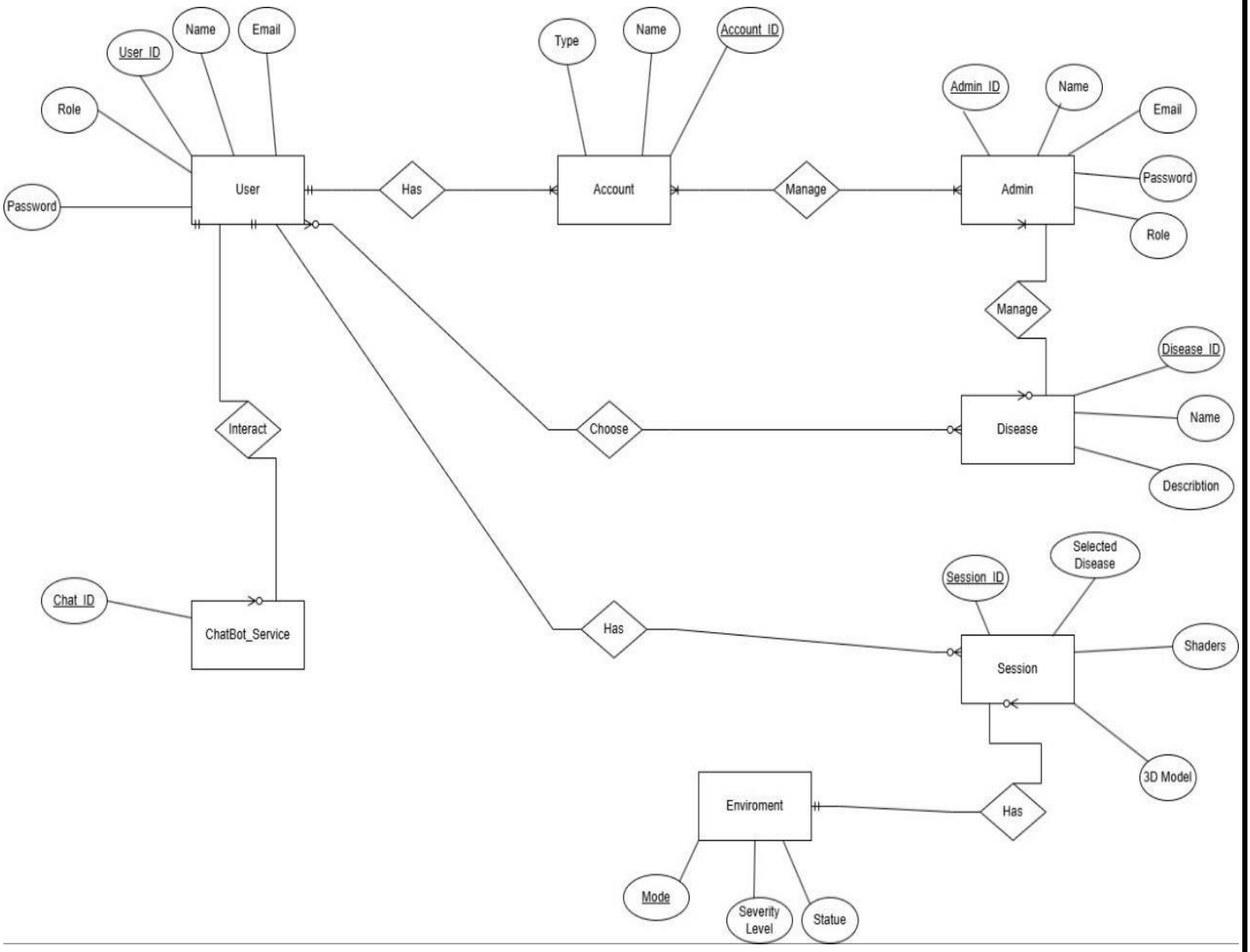
The project depends on:

- W3C WebXR and WebGL specifications
- Medical references (NIH, NEI, AAO) for disease descriptions
- Software engineering standards for safety and usability

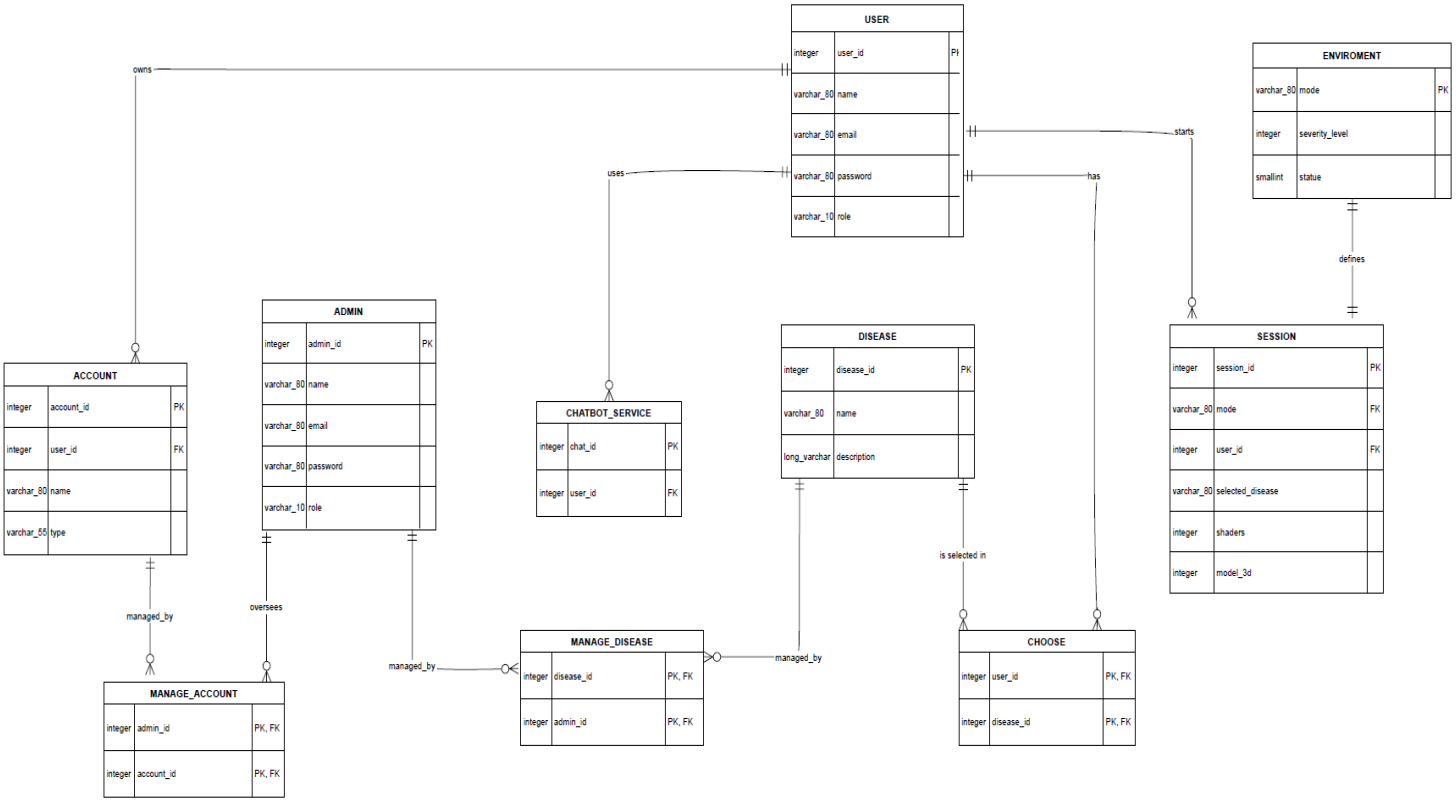
Changes in these standards may require updates to the system.

4.3 Design of Database ERD

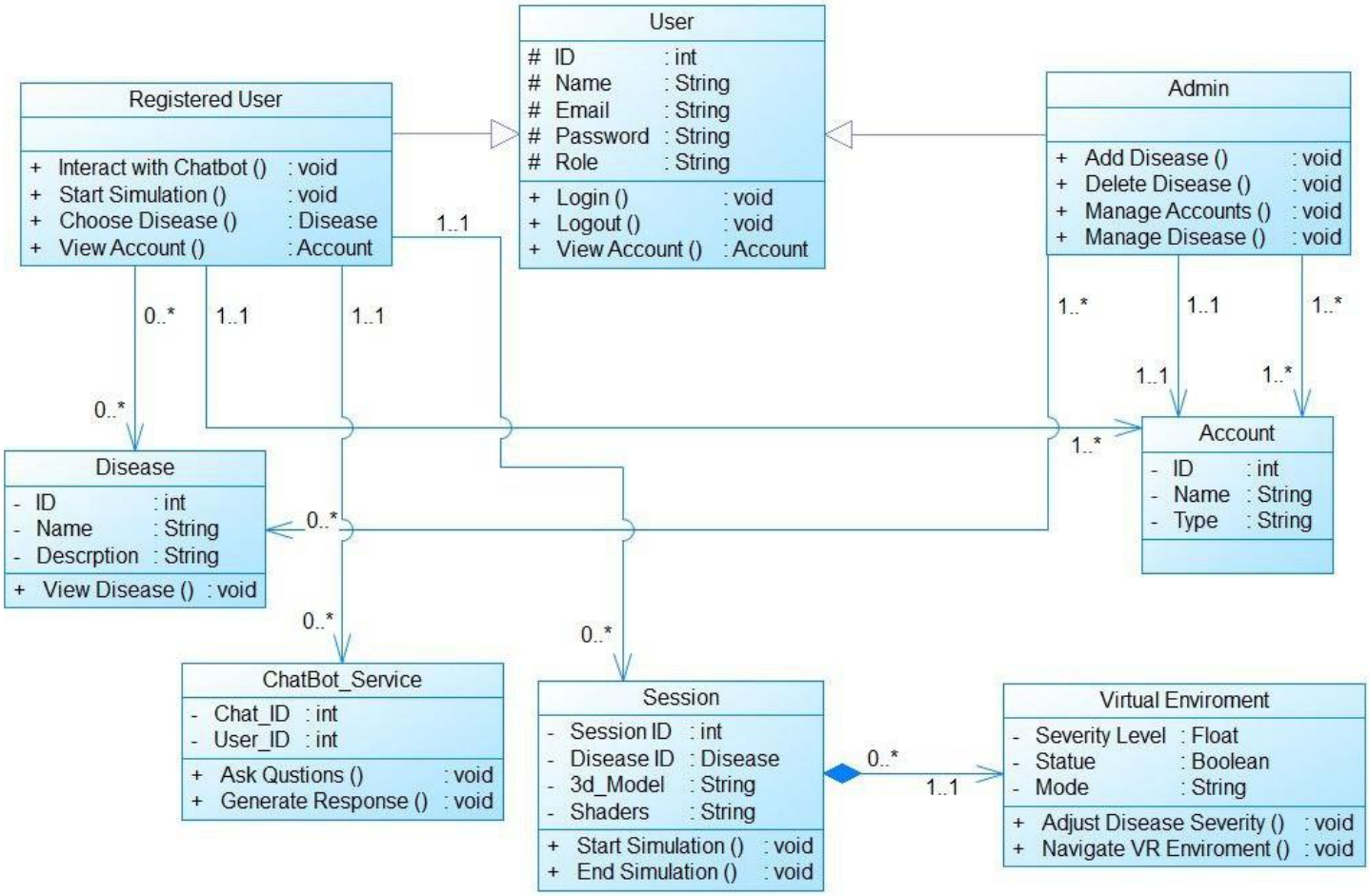
4.3.1 Entity Relationship Diagram



4.3.2 Mapping of ERD

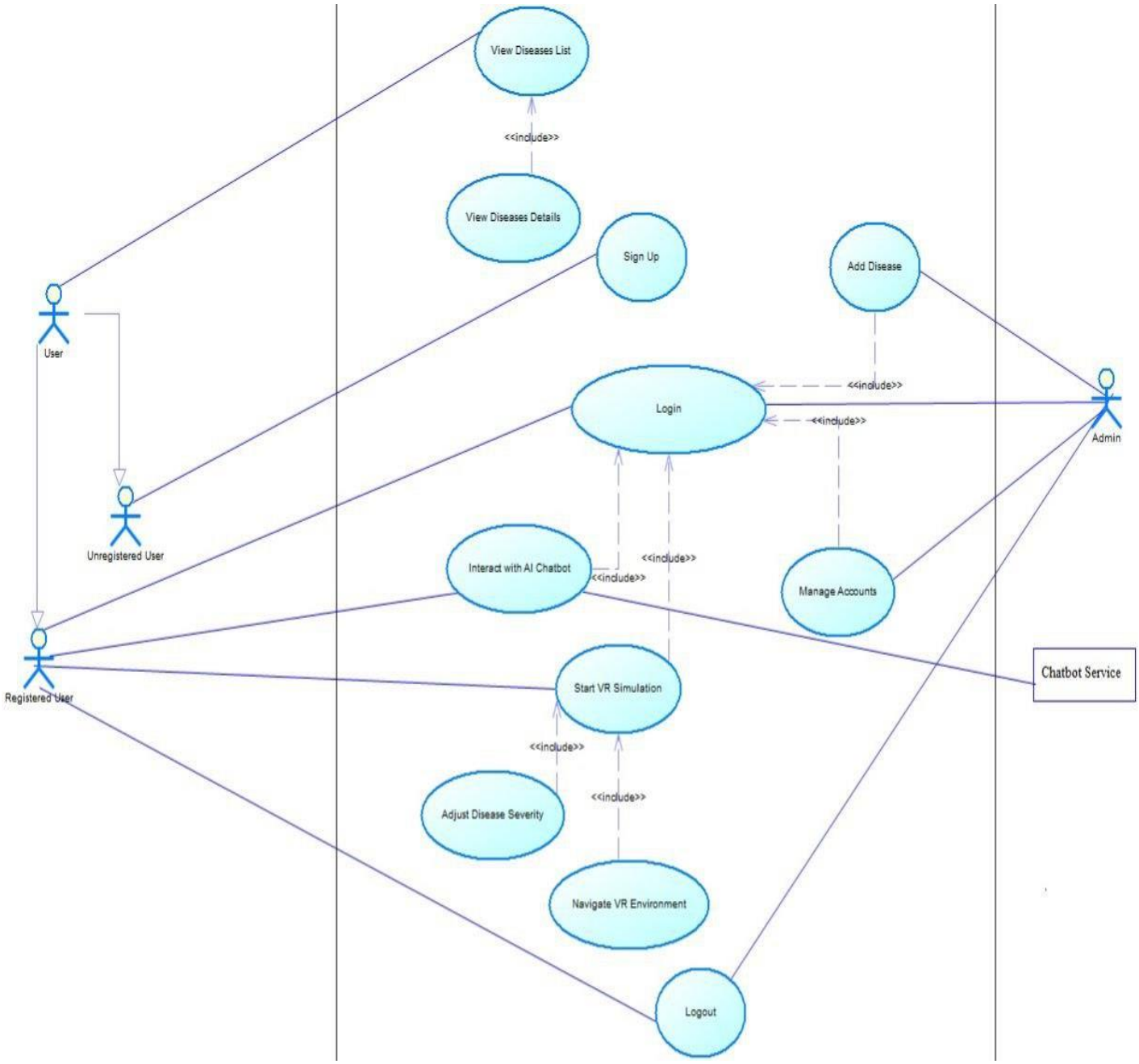


4.4 Class diagram



4.5 Use case Diagram

4.5.1 Primary Use-case diagram



4.5.2 Use-case scenarios

Table 8: UC-01

Use Case Number:	**UC-01**	
Use Case Name:	**Sign-Up**	
Actors:	**Unregistered User**	
Overview:	Allows an unregistered user to create a new account by providing required information.	
Related use cases:		
Event (Stimulus):	**User Action**	**System Action**
	1- Open Sign-Up page	**2.** Display sign-up form
	3. Fill in required fields	
	4. Submits the form.	**5.** System receives input and validates format. **6.** System checks if email already exists. **7.** System confirms successful registration.
	8. User sees success message	
		9. System redirects user to login page
Exceptions:	**Condition** **1.** Invalid input format	**System Response** **1.** Show validation error message.

	2. Passwords do not match **3.** Email already registered	**2.** Display "Passwords do not match" error. **3.** Show "Email already exists" error.
Comments:	Successful Sign-Up allows the user to proceed to Login (UC-02).	

(continued from previous page)

Table 9: UC-02

Use Case Number:	**UC-02**	
Use Case Name:	**Login**	
Actors:	**Registered User - Admin**	
Overview:	Allows a registered user / admin to access the system by entering valid login credentials.	
Related use cases:	**UC-01 Sign-Up** **UC-03 Logout** **UC-06 Interact with AI Chatbot** **UC-07 Start VR Simulation** **UC-08 Adjust Disease Severity** **UC-09 Navigate Virtual Environment** **UC-10 Manage Disease Content** **UC-11 Manage Accounts**	
Event (Stimulus):	**User Action**	**System Action**
	1. User opens the Login page.	**2.** System displays login form.
	3. User enters email and password.	

	4. User submits form.	**5.** System receives input and validates format. **6.** System checks if email exists. **7.** System verifies password against stored hash.
		8. User is redirected to home page.
Exceptions:	**Condition** **1.** Email incorrect **2.** Passwords incorrect **3.** Missing input	**System Response** **1.** Display "Invalid email or password" **2.** Display "Invalid email or password". **3.** Show validation error.
Comments:	Successful Login gives the user access to main system features.	

(continued from previous page)

Table 10: UC-03

Use Case Number:	**UC-03**	
Use Case Name:	**Logout**	
Actors:	**Registered User - Admin**	
Overview:	Allows a logged-in user to exit the system.	
Related use cases:	**UC-02 Login**	
Event (Stimulus):	**User Action**	**System Action**
	1- User clicks "Logout" button.	**2.** System receives logout request.
		3. System ends session
		4. User is redirected to the Login page.
Exceptions:	**Condition**	**System Response**
Comments:	Logout from the system so no further access to chatbot and simulation.	

Table 11: UC-04

Use Case Number:	**UC-04**	
Use Case Name:	**View Diseases List**	
Actors:	**User**	
Overview:	Allows the user to view a list of all available retinal diseases in the system and choose the preferred one to view detailed educational information	
Related use cases:	**UC-05 View Disease Details**	
Event (Stimulus):	**User Action**	**System Action**
	1. User navigates to the "Diseases" page.	**2.** System retrieves list of diseases and display them
	3. User browses the list.	
	4. User selects a disease from the disease list.	
Exceptions:	**Condition**	**System Response**
Comments:	Provides the user with list of diseases to view detailed information about a selected retinal disease.	

Table 12: UC-05

Use Case Number:	**UC-05**	
Use Case Name:	**View Disease Details**	
Actors:	**User**	
Overview:	Allows the user to view detailed information about a selected retinal disease.	
Related use cases:	**UC-04 View Diseases List**	
Event (Stimulus):	**User Action**	**System Action**
	1. User selects a disease from the disease list.	**2.** System displays detailed information (description, symptoms, and causes).
	3. User reads the disease information.	
Exceptions:	**Condition**	**System Response**
Comments:	Viewing disease details helps users understand each disease before starting a VR simulation.	

Table 13: UC-06

Use Case Number:	**UC-06**	
Use Case Name:	**Interact with AI Chatbot**	
Actors:	**Registered User**	
Overview:	Allows the user to ask questions about diseases and receive AI-generated response.	
Related use cases:	**UC- 02 Login**	
Event (Stimulus):	**User Action**	**System Action**
	1. User opens the chatbot panel.	**2.** System loads the chatbot interface.
	3. User types a question and sends it.	**4.** System receives message. **5.** System forwards the question to the AI chatbot service (OpenAI API). **6.** System receives AI-generated response. **7.** System displays the chatbot reply in the conversation window.
	8. User reads the response.	
Exceptions:	**Condition** **1.** No internet / API unreachable	**System Response** **1.** Show "Chatbot currently unavailable. Try again later."

Comments:	Chatbot provides real-time explanations and guidance related to retinal diseases.

(continued from previous page)

Table 14: UC-07

Use Case Number:	**UC-07**	
Use Case Name:	**Start VR Simulation**	
Actors:	**Registered User**	
Overview:	Allows the user to start a VR simulation for a selected retinal disease.	
Related use cases:	**UC-02 Login** **UC-04 View Disease List**	
Event (Stimulus):	**User Action**	**System Action**
	1. selects "Start Simulation" on a disease simulation page.	**2.** System creates a new simulation session. **3.** System loads the virtual environment and prepares assets (3D model, shader, textures).
	4. User enters VR mode or 3D simulation mode.	**5.** System initializes rendering and displays the disease visual effect.
Exceptions:	**Condition**	**System Response**

	1. VR not supported on device	**1.** Display fallback 3D preview mode.
Comments:	Successful simulation launch allows the user to adjust severity (UC-08).	

(continued from previous page)

Table 15: UC-08

Use Case Number:	**UC-08**	
Use Case Name:	**Adjust Disease Severity**	
Actors:	**Registered User**	
Overview:	Allows the user to adjust the severity of the selected retinal disease during the simulation by moving a slider or control.	
Related use cases:	**UC-07 Start VR Simulation**	
Event (Stimulus):	**User Action**	**System Action**
	1. User moves the severity slider during simulation.	**2.** System reads the new severity value.
		3. System updates shader parameters based on severity. **4.** System re-renders the virtual environment with updated distortion effects

		5. System applies real-time rendering changes continuously.
	6. User observes the updated visual effect.	
Exceptions:	**Condition** **1.** Invalid slider value **2.** Rendering failure	**System Response** **1.** Clamp severity value to valid range. **2.** Display error and fallback to default severity.
Comments:	Providing real-time visualization of disease progression inside the VR simulation.	

(continued from previous page)

Table 16: UC-09

Use Case Number:	**UC-09**	
Use Case Name:	**Navigate Virtual Environment**	
Actors:	**Registered User**	
Overview:	Allows the user to move and look around inside the VR/3D environment while a simulation is running.	
Related use cases:	**UC-07 Start VR Simulation** **UC-08 Adjust Disease Severity**	
Event (Stimulus):	**User Action**	**System Action**
	1. User enters VR/3D simulation mode.	**2.** System initializes locomotion and camera controls.
	3. User moves (walk/teleport) or looks around using controllers/mouse/keyboard.	**4.** System updates user position and camera orientation in the scene.
		5. System continues rendering scene from updated viewpoint.
Exceptions:	**Condition** **1.** Input device not detected **2.** Collision or invalid move	**System Response** **1.** Show control hints and offer fallback controls (keyboard/mouse).

		2. Prevent movement and snap to nearest valid position.
Comments:	Navigation should be smooth and comfortable to prevent motion sickness; provide clear control instructions and a fallback non-VR preview for unsupported devices.	

(continued from previous page)

Table 17: UC-10

Use Case Number:	**UC-10**	
Use Case Name:	**Add Disease**	
Actors:	**Admin**	
Overview:	Allows the Admin to manage content.	
Related use cases:	**UC – 02 Login**	
Event (Stimulus):	**User Action**	**System Action**
	1. Admin opens the Add Disease" page.	**2.** System displays fields for list setup.
	3. Admin adds/edits/deletes disease	**4.** System validates content
		5. System saves updates to database

		6. System confirms successful update
Exceptions:	**Condition** **1.** Invalid data input	**System Response** **3.** Show validation error message
Comments:	This use case deals with content setup.	

(continued from previous page)

Table 18: UC-11

Use Case Number:	**UC-11**	
Use Case Name:	**Manage Accounts**	
Actors:	**Admin**	
Overview:	Allows the Admin to view, update user accounts in the system.	
Related use cases:	**UC-02 Login**	
Event (Stimulus):	**User Action**	**System Action**
	1. Admin opens the "Manage Accounts" page.	**2.** System retrieves all user accounts from the database.
	3. Admin selects a user account.	**4.** System loads account details.
	5. Admin chooses an action (update role, deactivate, activate, delete).	**6.** System validates the requested action.
	6. Admin confirms the change.	**7.** System updates the user record in the database and confirms

		the account update to the admin.
Exceptions:	**Condition** **1.** Invalid input (e.g., invalid role)	**System Response** **1.** Show validation error message.
Comments		

(continued from previous page)

4.6 Activity Diagram

Figure 13: Sign Up (User)

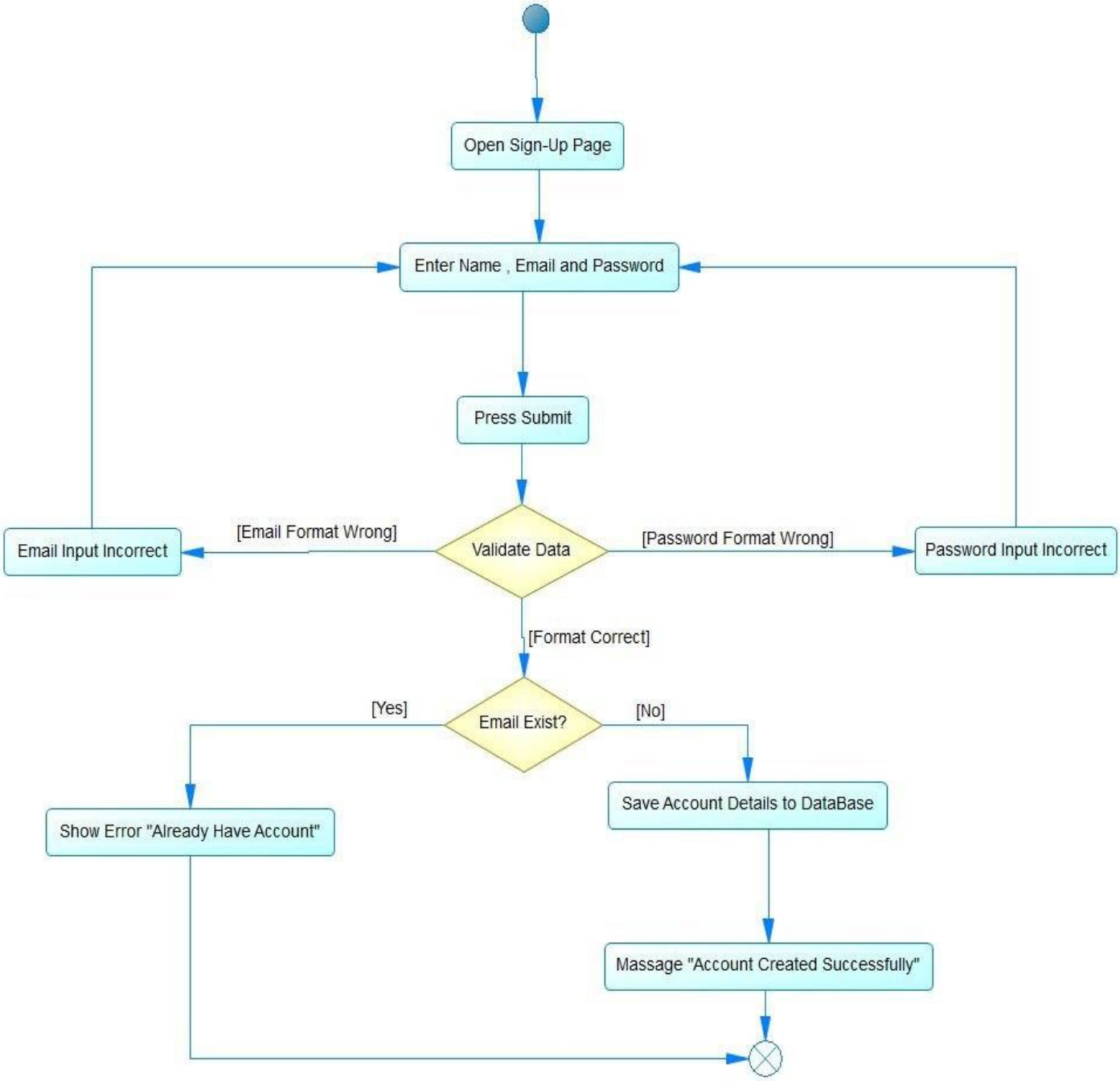


Figure 14: Login, Logout (User, Admin)

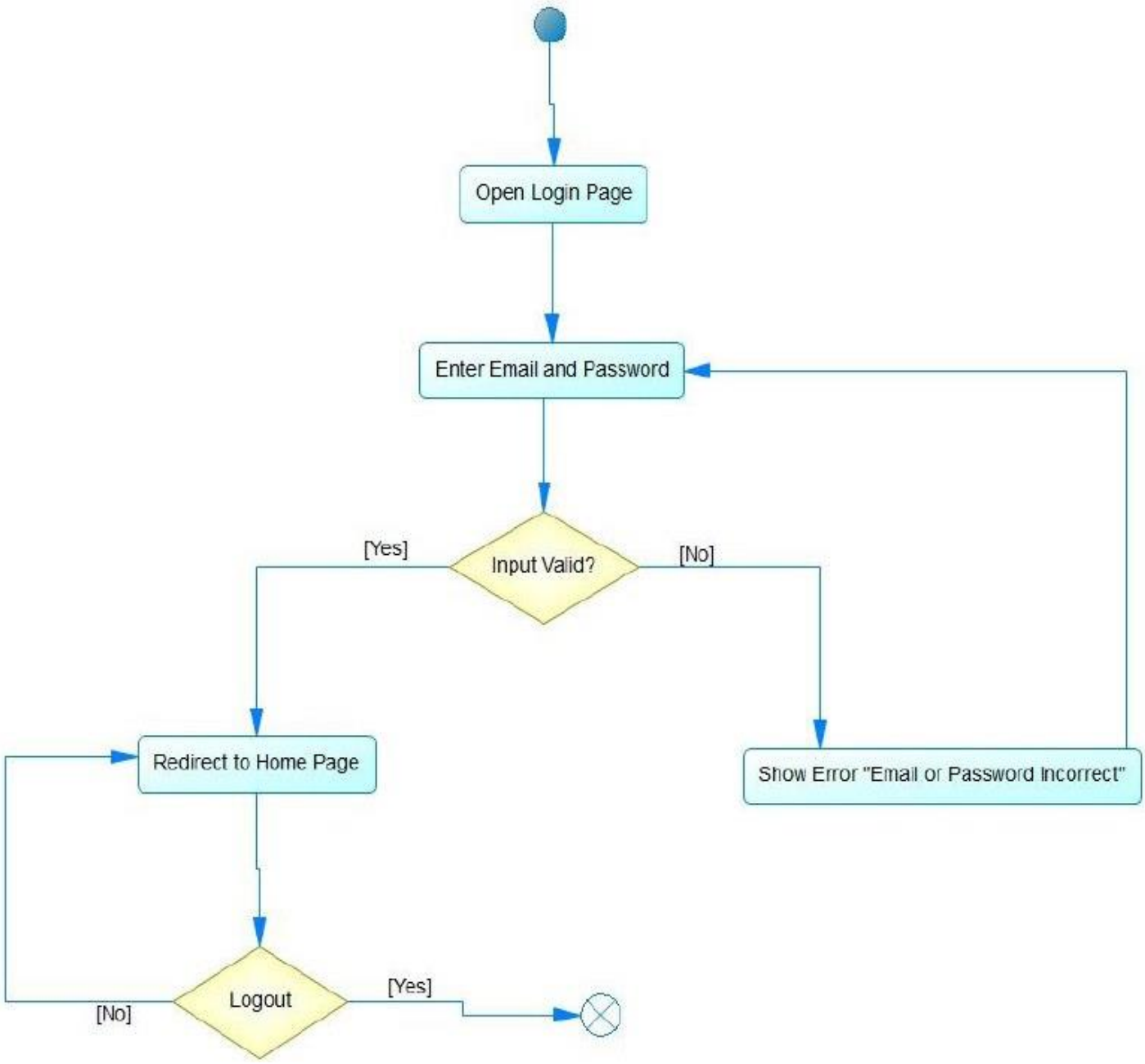


Figure 15: View Diseases Educational Info (User)

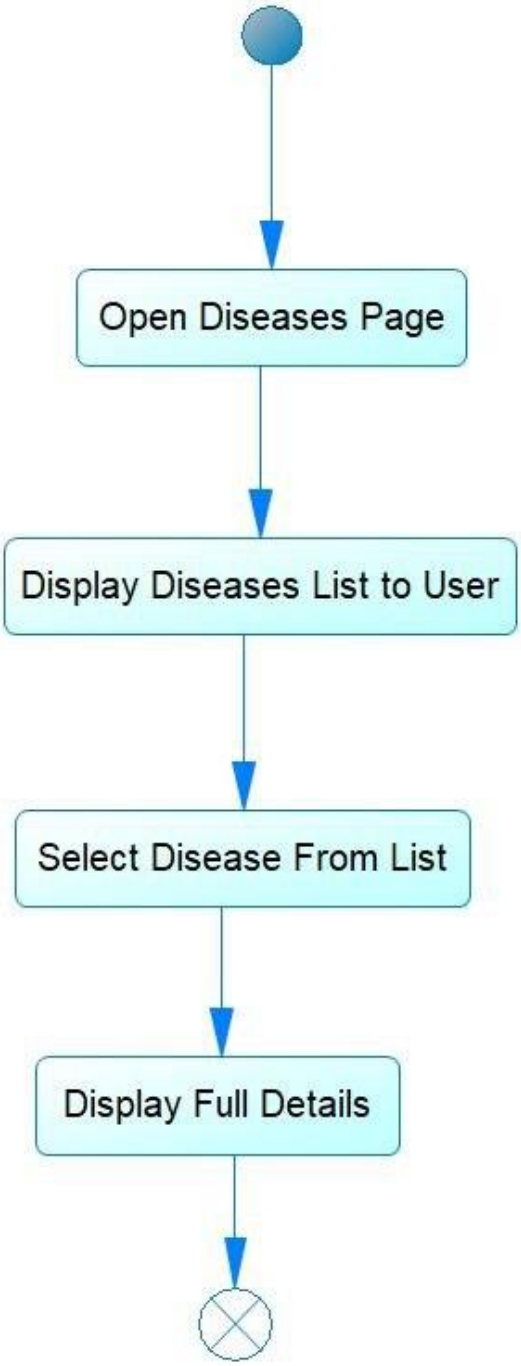


Figure 16:Start, Navigate, and control VR simulation (User)

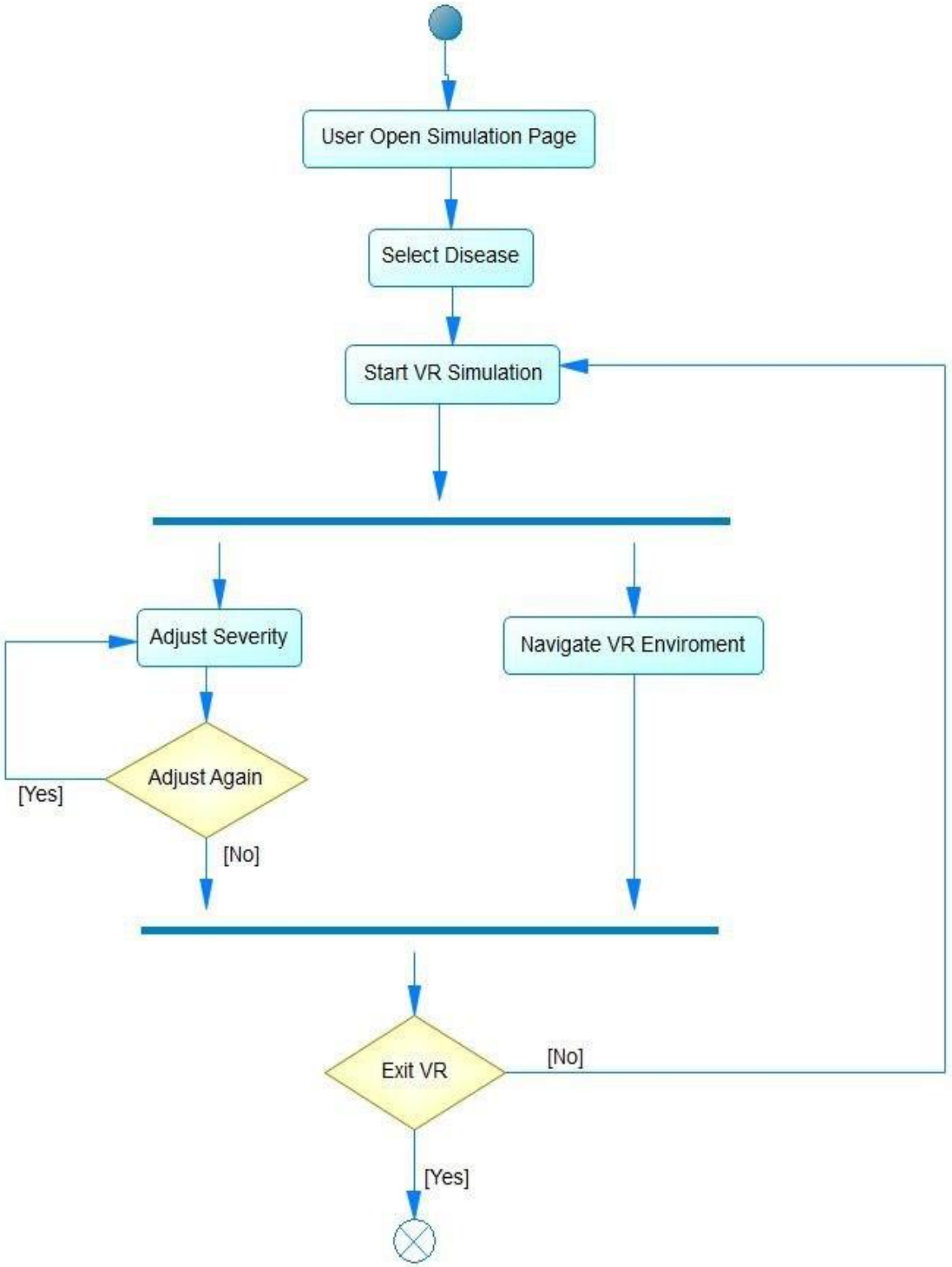


Figure 17:Interact with AI Chatbot (User)

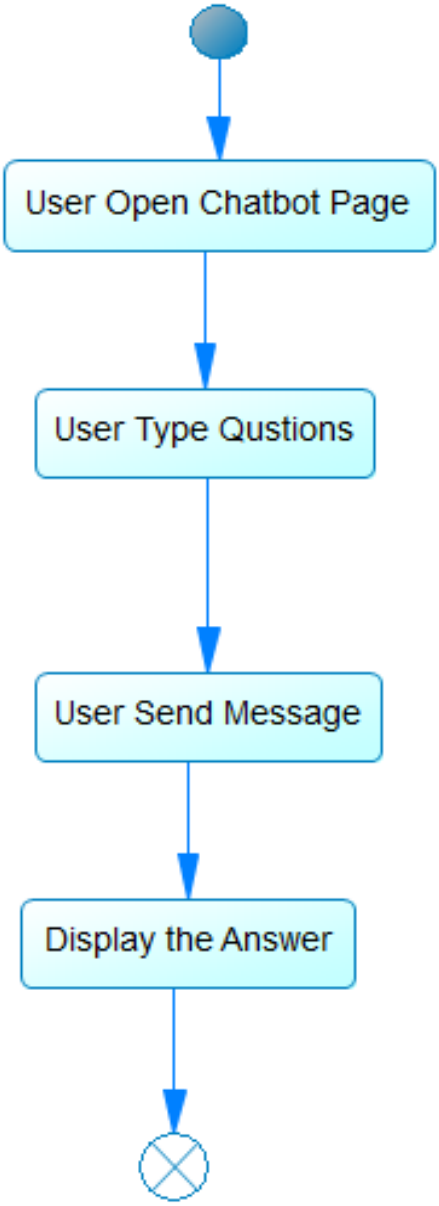


Figure 18: Add Disease (Admin)

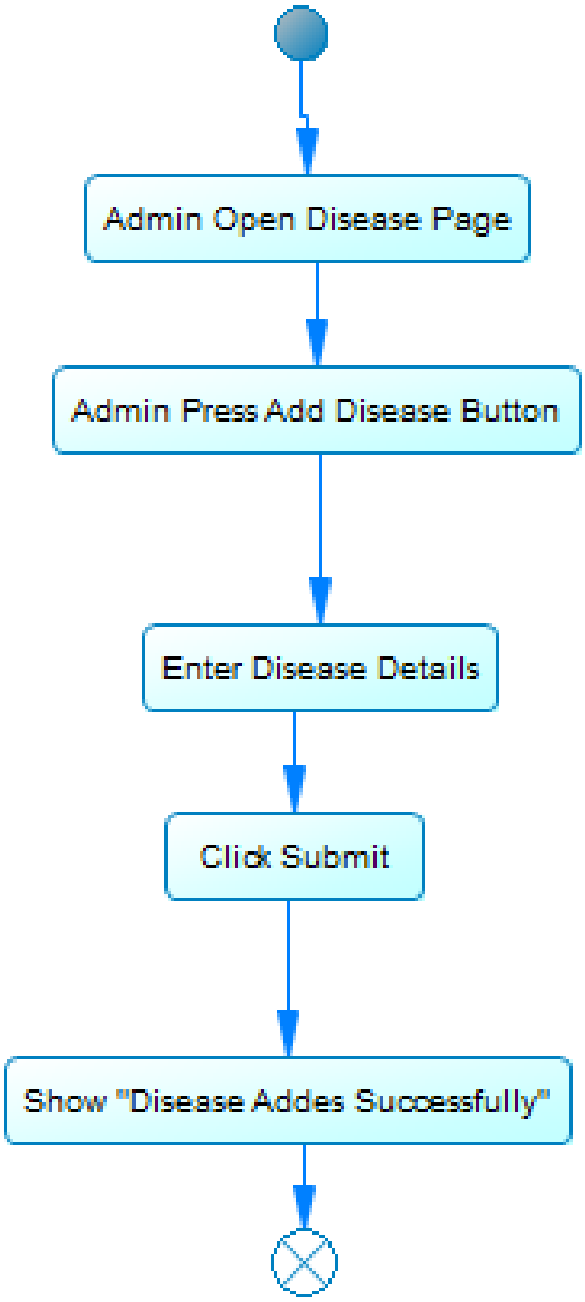
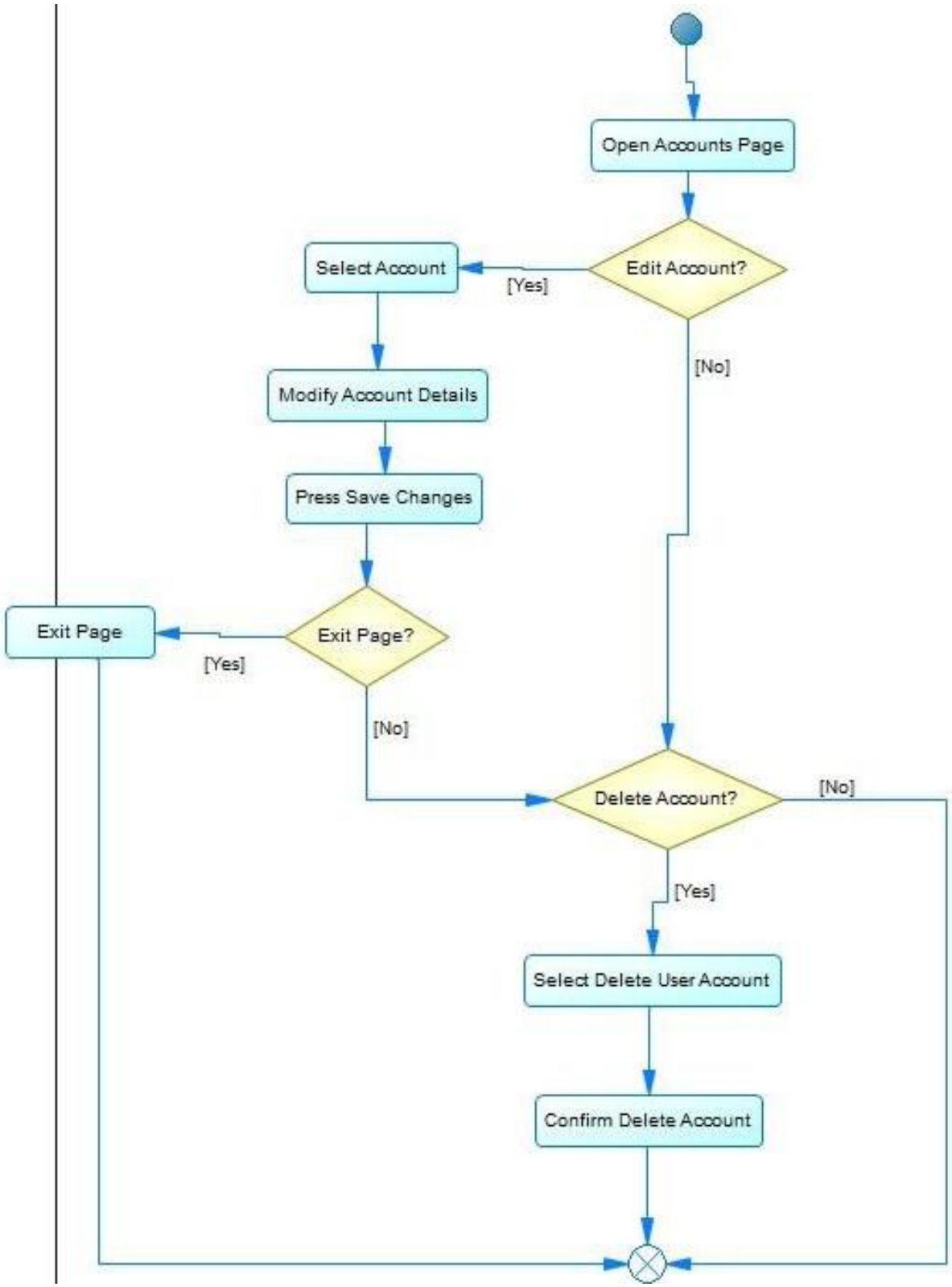


Figure 19: Manage accounts (Admin)



4.7 Sequence Diagram

Figure 20: Signup (User)

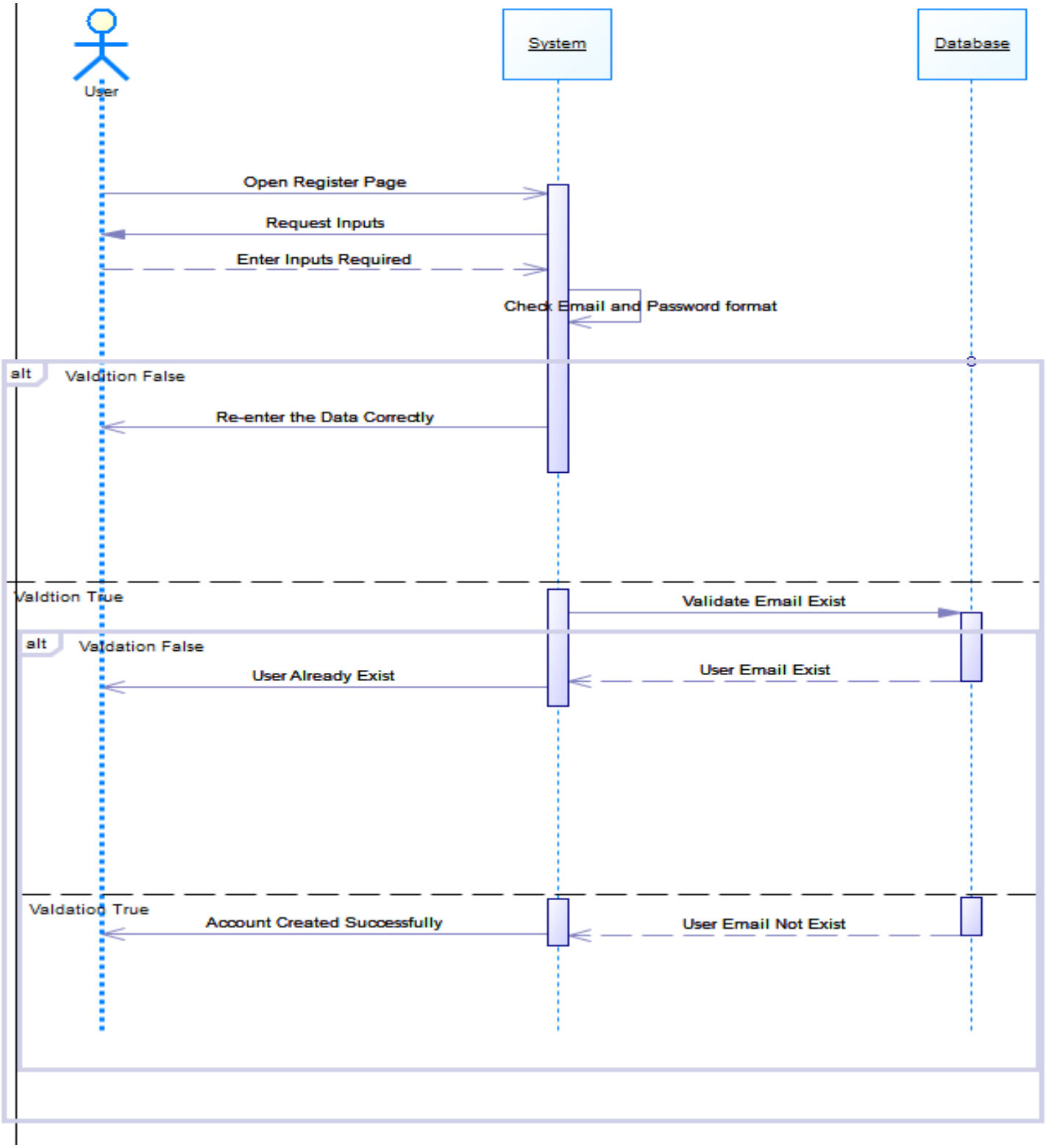


Figure 21: Login (User)

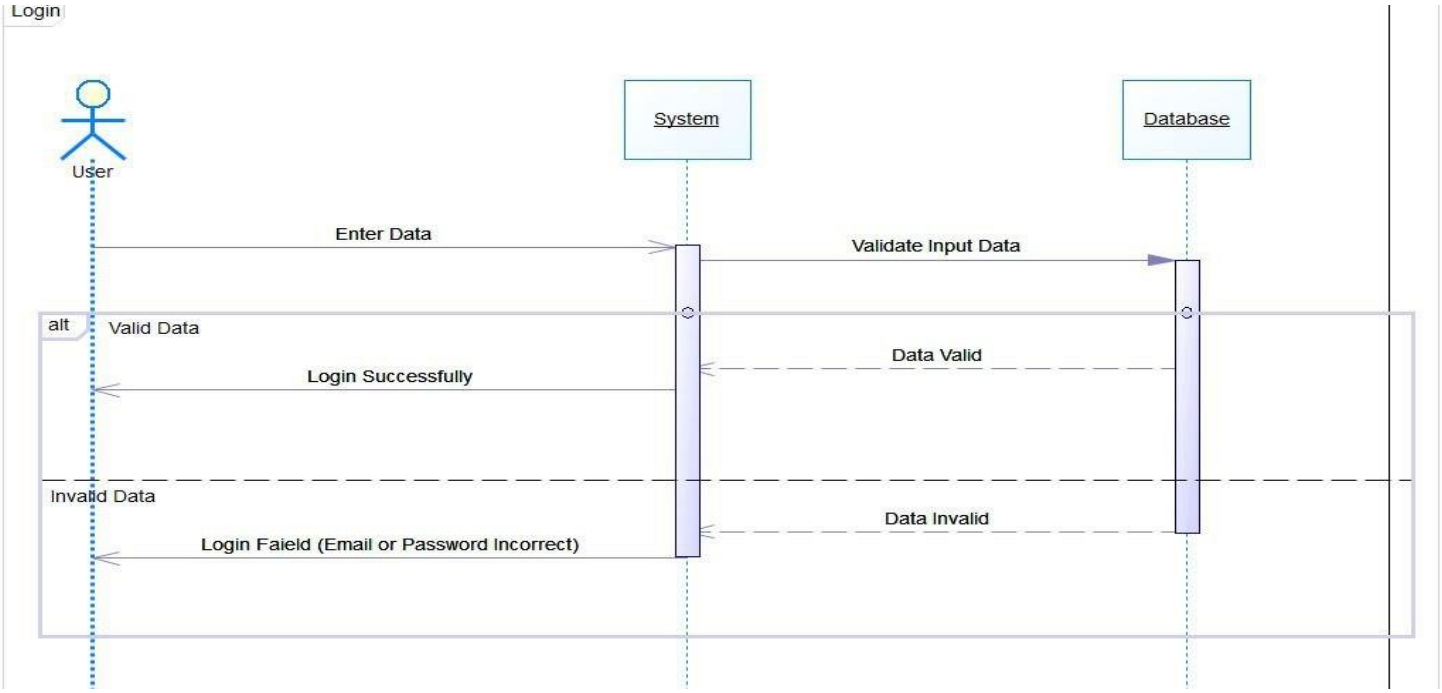


Figure 22: Login (Admin)

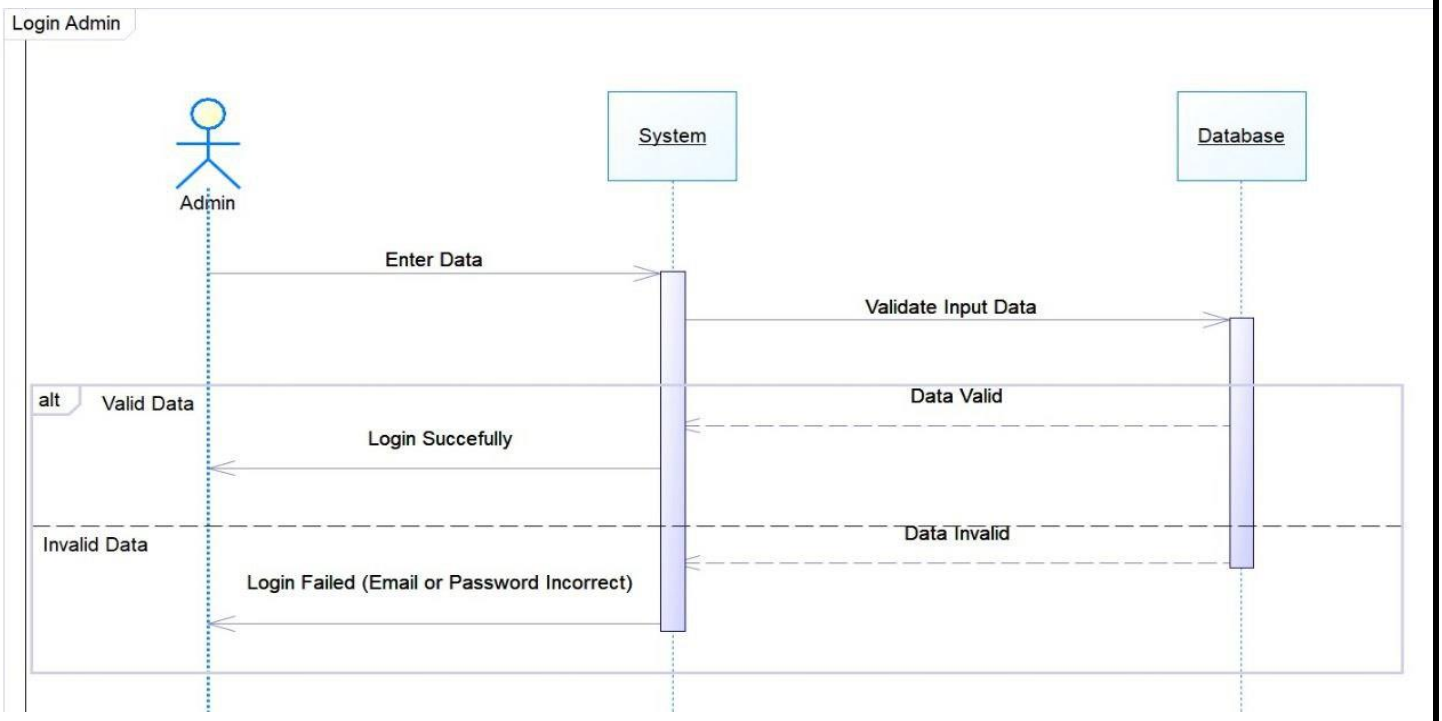


Figure 23: Logout (User)

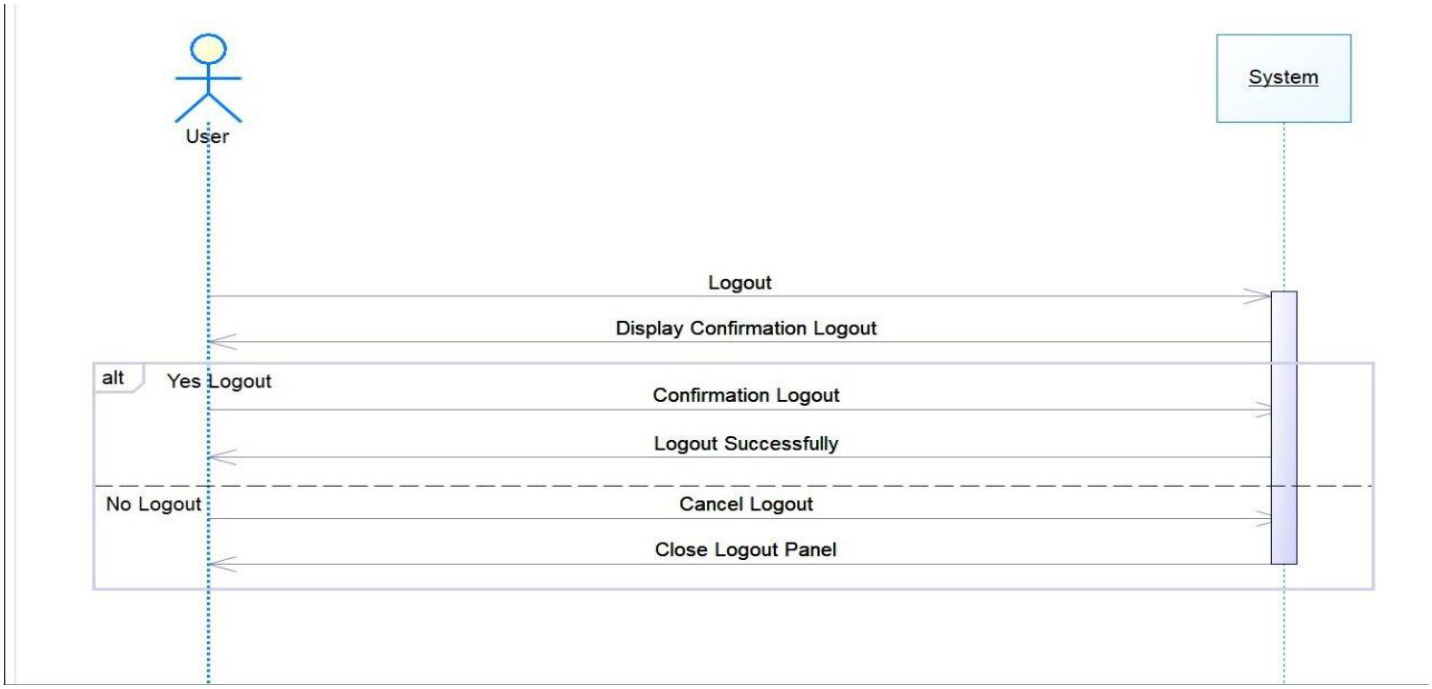


Figure 24: Logout (Admin)

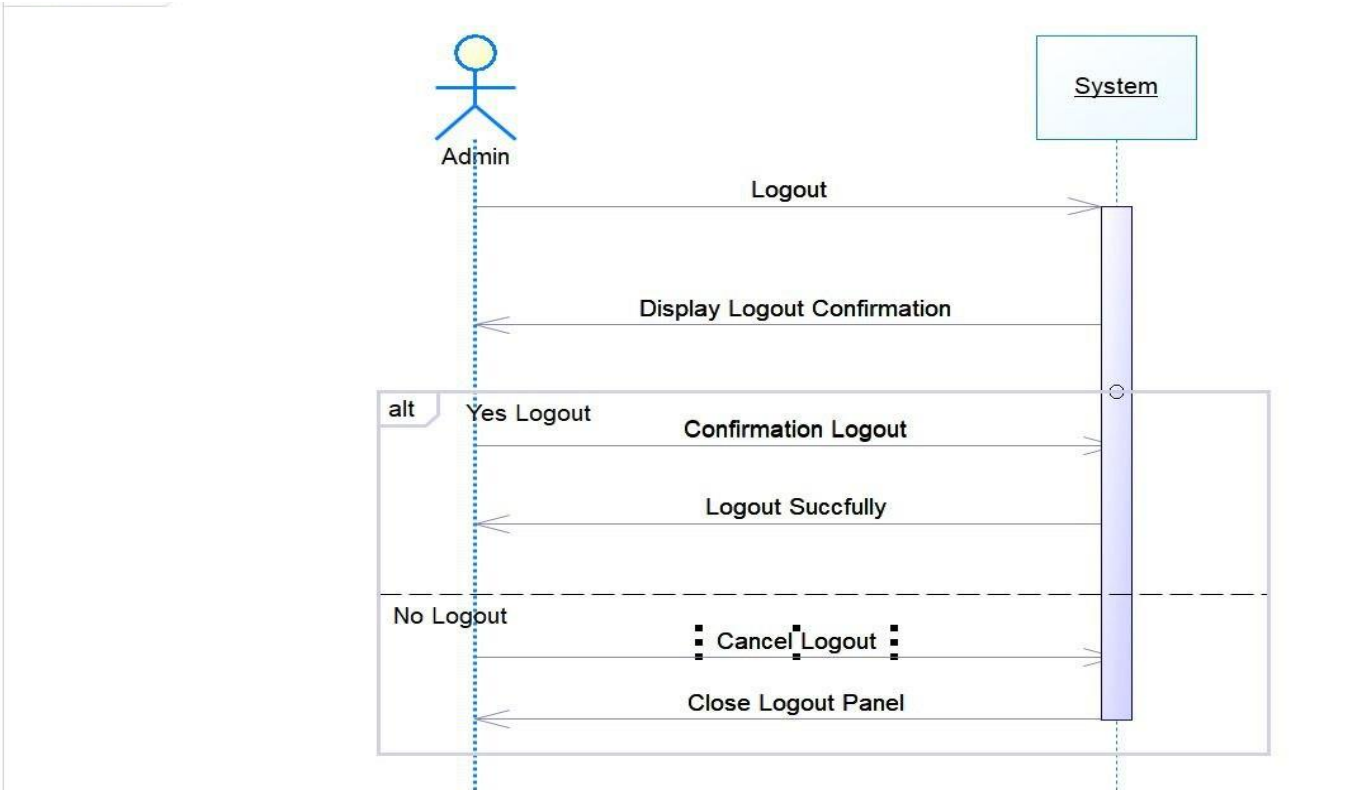


Figure 25: View diseases List (User)

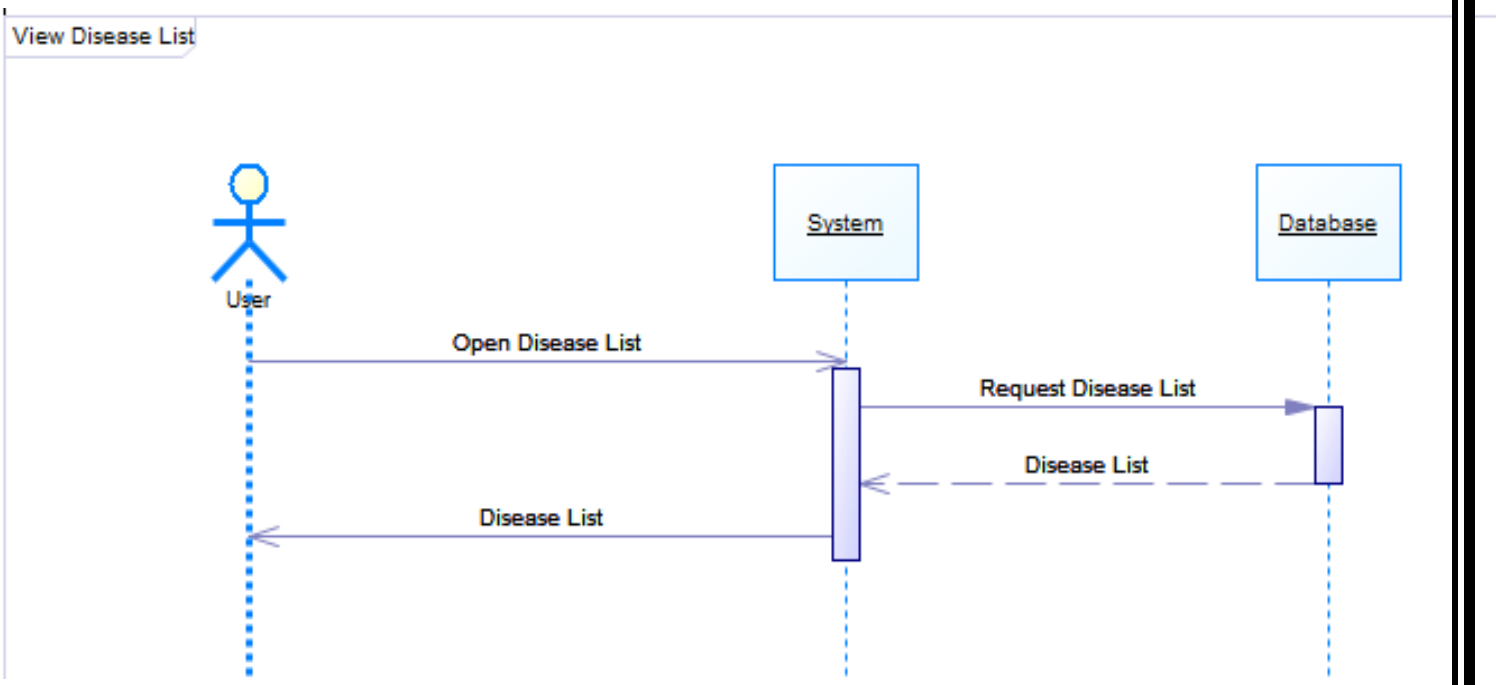


Figure 26: View diseases details (User)

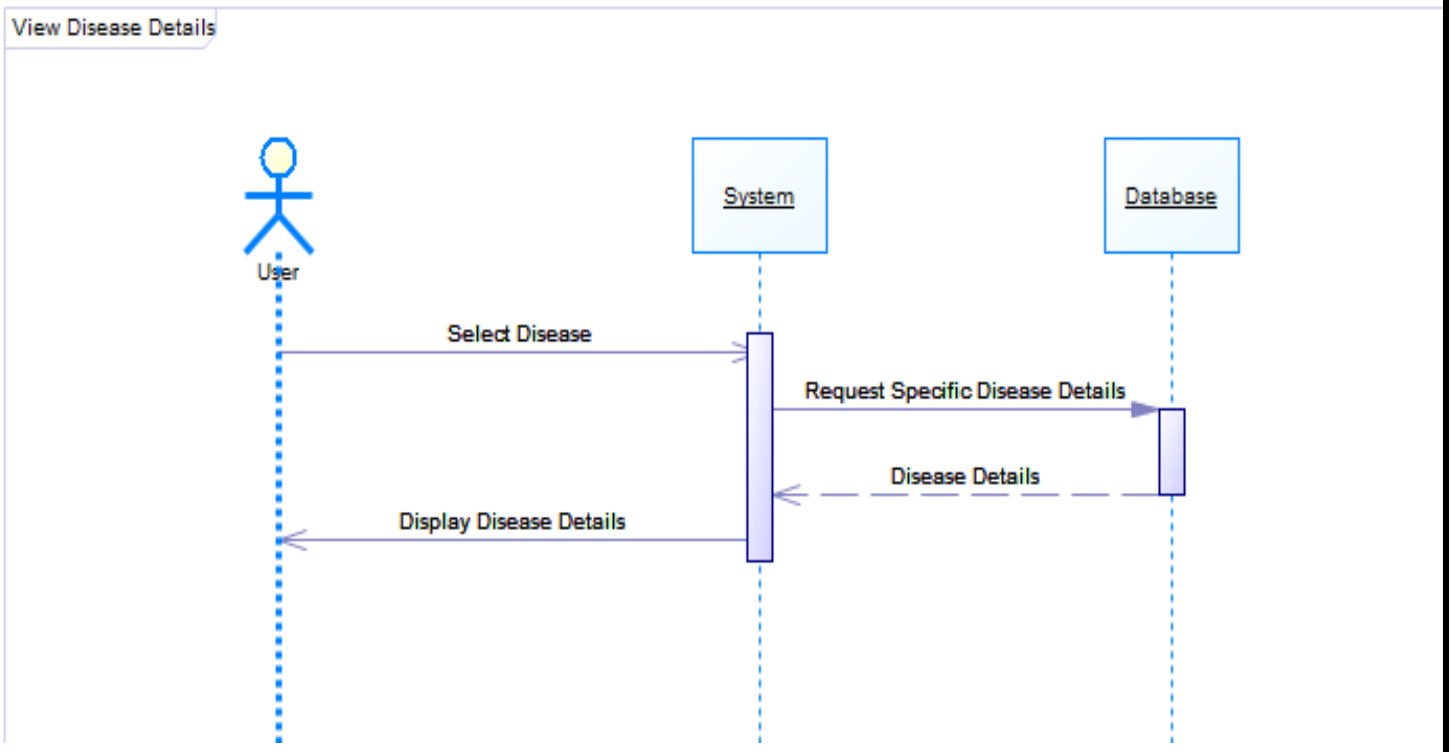


Figure 27: Interact with AI chatbot (User)

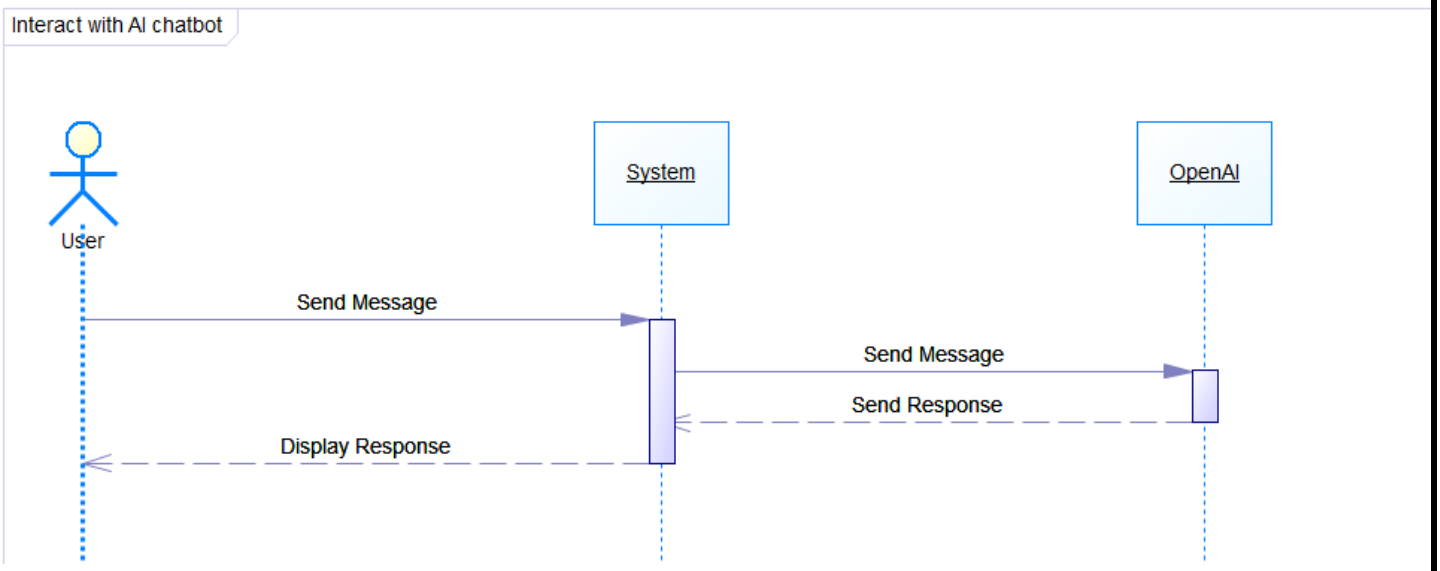


Figure 28: Start VR Simulation

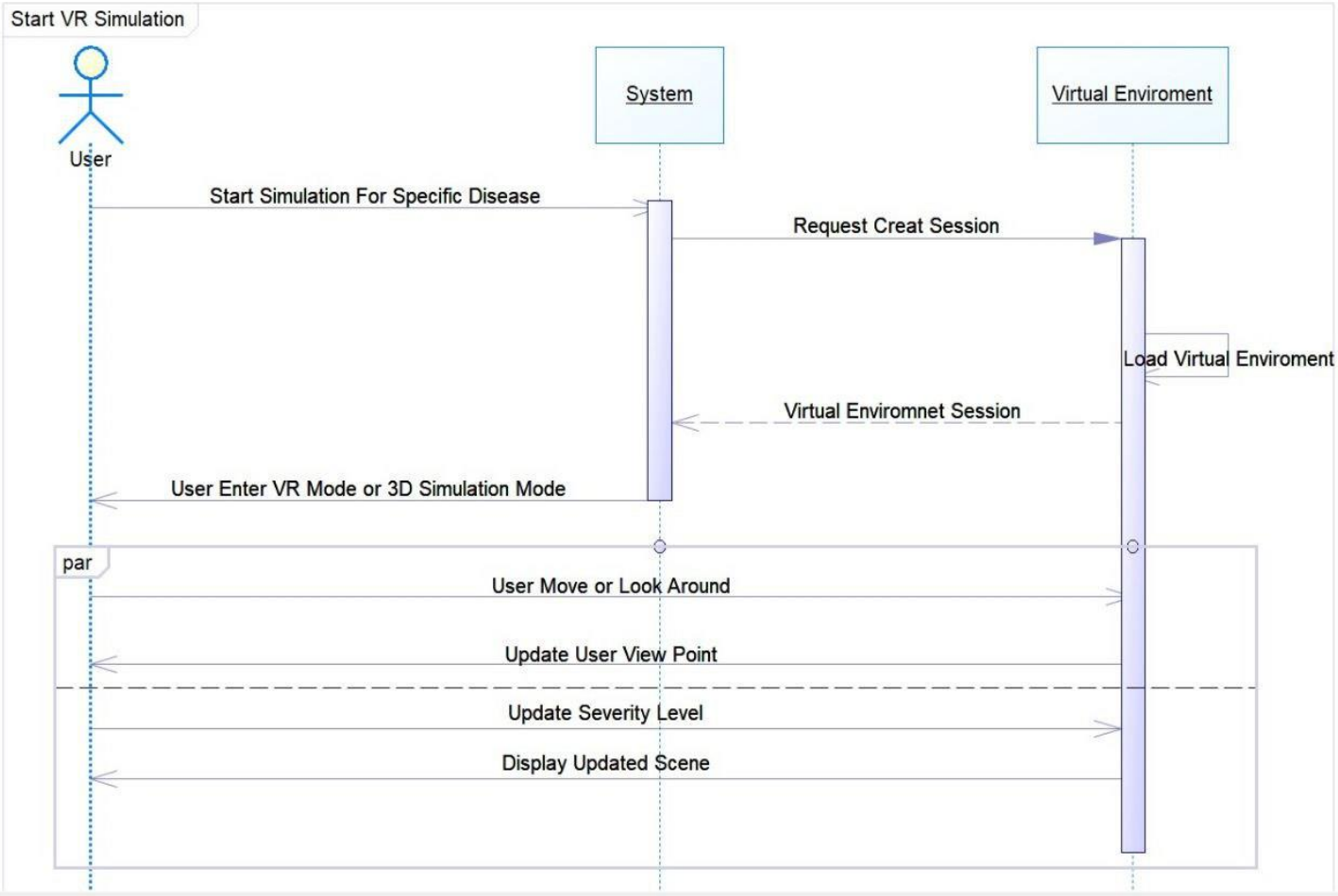


Figure 29: Manage accounts (Admin)

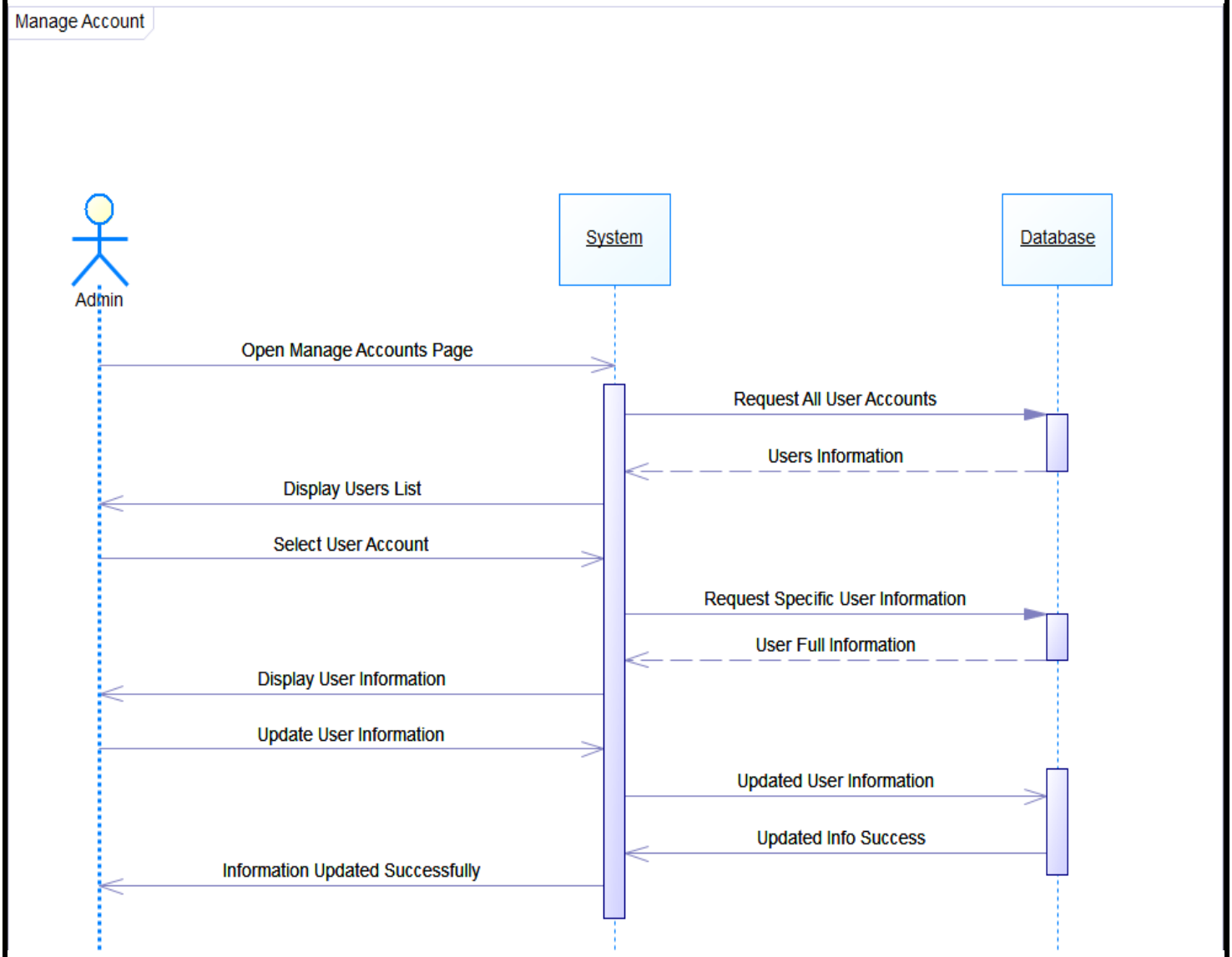
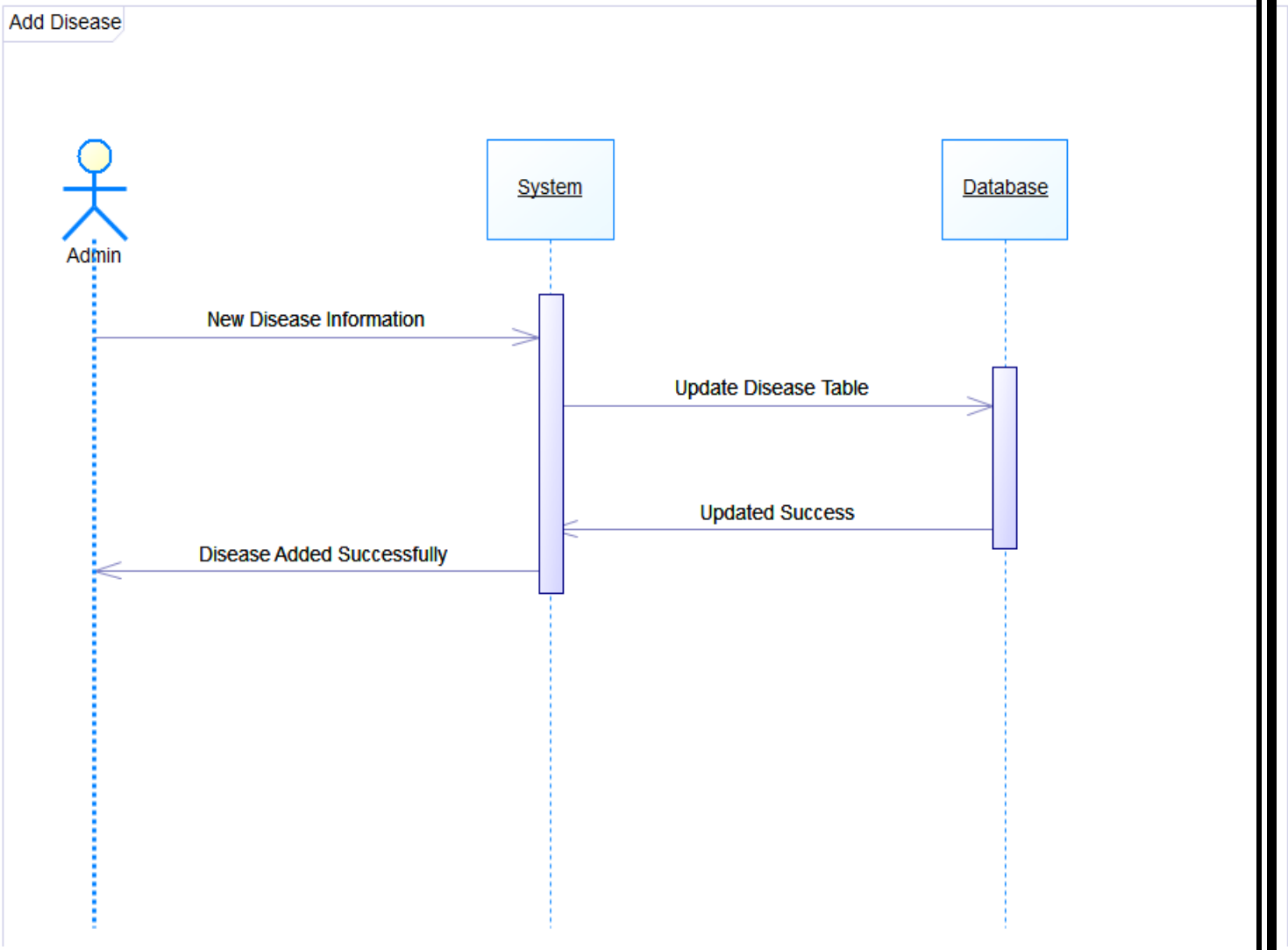


Figure 30: Add Disease (Admin)



4.8 State Diagram

Figure 31: Simulation

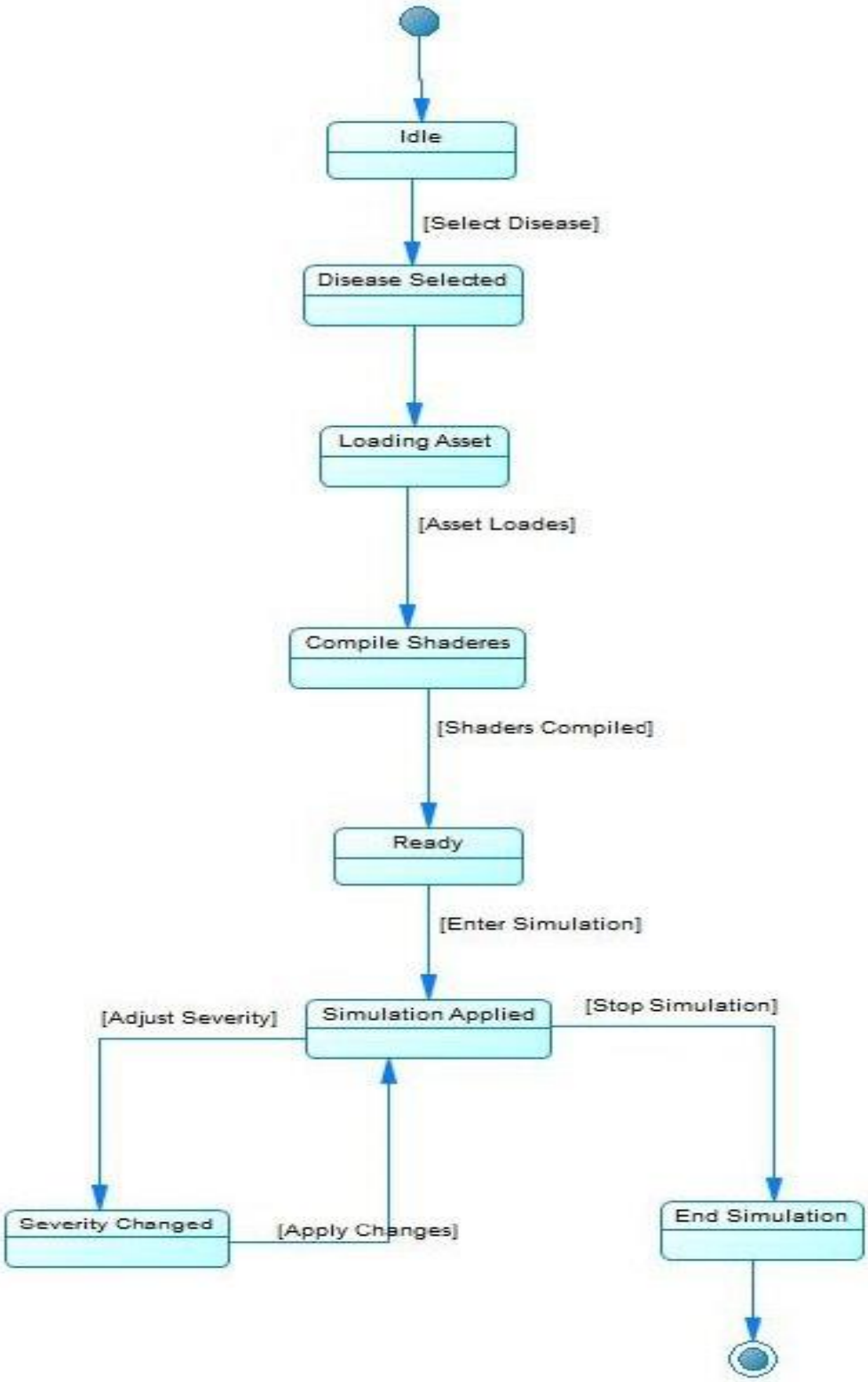
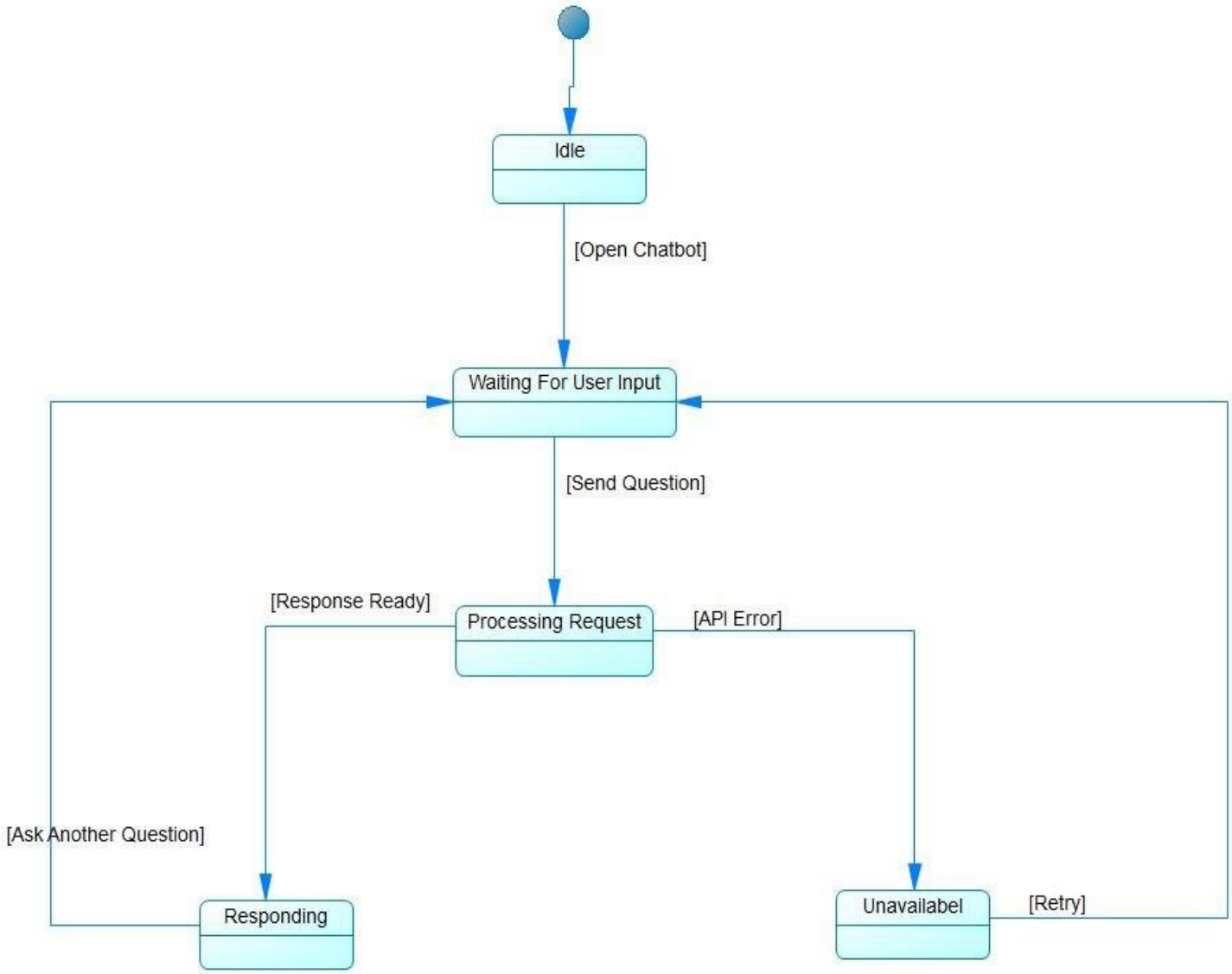


Figure 32: Chatbot



References

[1] VRVis. (n.d.). *XREye: Simulating visual impairments in virtual and augmented reality*. Retrieved from https://www.vrvis.at/en/research/research-projects/xreye

[2] ResearchGate. (2023). *Visual impairment simulation in virtual reality: (A) age-related macular degeneration, (B) cataracts, and (C) glaucoma*. Retrieved from https://www.researchgate.net/figure/Visual-impairment-simulation-in-virtual-reality-A-age-related-macular-degeneration_fig1_371729893

[3] EyeNews. (2023). *Simulating the visual impairment symptoms of age-related macular degeneration in virtual reality*. Retrieved from https://www.eyenews.uk.com/features/ophthalmology/post/simulating-the-visual-impairment-symptoms-of-age-related-macular-degeneration-in-virtual-reality

[4] S. H. Al-Samarraie, et al. (2023). A virtual reality-based application for simulating visual impairments: Usability and effectiveness study. *PLOS Digital Health*, 2(6), e0000277. Retrieved from https://pmc.ncbi.nlm.nih.gov/articles/PMC10281573/

[5] EmpathEyes. (n.d.). *EmpathEyes Vision Simulation*. Retrieved from https://www.empatheyesvision.com/

[6] National Eye Institute (NEI). (n.d.). *See What I See: Virtual Reality Eye Disease Experience*. Retrieved from https://www.nei.nih.gov/learn-about-eye-health/outreach-resources/see-what-i-see-virtual-reality-eye-disease-experience

[7] Zwoliński, G., Neugebauer, A., Severitt, B., et al. (2025). *Visual impairments simulation in virtual reality as an empathy booster*. Springer. Retrieved from https://link.springer.com/article/10.1007/s10055-025-01150-z

[8] Neugebauer, A., Castner, N., Wahl, S., et al. (2024). *Simulating vision impairment in virtual reality: tunnel vision conditions*. Springer. Retrieved from https://link.springer.com/article/10.1007/s10055-024-00987-0

[9] Jones, P. R., et al. (2020). *Evaluating gaze-contingent visual impairment simulation in VR*. npj Digital Medicine. Retrieved from https://pmc.ncbi.nlm.nih.gov/articles/PMC7064490/

[10] Bennett, J. et al. (2023). *Optimization and validation of a VR orientation & mobility (O&M) test for inherited retinal degenerations*. PMC.
Retrieved from https://pmc.ncbi.nlm.nih.gov/articles/PMC9896841/

[11] Lam, A. K. N., et al. (2020). *Use of virtual reality simulation to identify vision-related disability in patients with glaucoma*. PMC.
Retrieved from https://pmc.ncbi.nlm.nih.gov/articles/PMC7225912/

[12] Masoudi, K. (2025). *A rapid review of the use of VR as a tool in vision care*. *Technologies*.
Retrieved from https://www.mdpi.com/2227-7080/13/8/342

[13] Hosp, B. W. (2024). *VisionaryVR: An optical simulation tool for evaluating vision impairments in VR*. University College London.
Retrieved from https://discovery.ucl.ac.uk/id/eprint/10191753/1/sensors-24-02458-v2.pdf

[14] Three.js Documentation (n.d.). *PostProcessing*.
Retrieved from https://threejs.org/docs/pages/PostProcessing.html

[15] Three.js Documentation (n.d.). *EffectComposer (post-processing chain)*.
Retrieved from https://threejs.org/docs/pages/EffectComposer.html

[16] OpenAI. (n.d.). *Chat completion and conversational AI*.
Retrieved from https://platform.openai.com/docs/guides/chat

[17] Mozilla Developer Network (MDN). (n.d.). *WebXR Device API*.
Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/WebXR_Device_API

[18] Jerald, J. (2015). *The VR book: Human-centered design for virtual reality*. ACM.
Retrieved from https://dl.acm.org/doi/book/10.1145/2792790

[19] VisionAid. (n.d.). *Eye Disease Simulator*.
Retrieved from https://www.visionaid.io/eye-disease-simulator

[20] National Eye Institute (NEI). (n.d.). *See What I See: Virtual Reality Eye Disease Experience*.

Retrieved from https://www.nei.nih.gov/about/education-and-outreach/see-what-i-see-virtual-reality-eye-disease-experience

[21] National Eye Institute (NEI). (2020). *See What I See: Virtual Reality Eye Disease Experience*.
Retrieved from https://www.nei.nih.gov/learn-about-eye-health/outreach-campaigns-and-resources/see-what-i-see-virtual-reality-eye-disease-experience

[22] Prevent Blindness. (2020). *Vision Programs – The Augmented Reality Eye App*.
Retrieved from https://ohio.preventblindness.org/the-eye-and-how-we-see/

[23] Tengo Baja Visión. (2017). *Simulator App: I Have Low Vision*.
Retrieved from https://www.tengobajavision.com/en/app/simulator/

[24] Varjo. (2022). *Case Study: How Lucid Reality Labs and Roche Raise Awareness for Macular Disease with Mixed Reality*.
Retrieved from https://varjo.com/case-studies/how-lucid-reality-labs-and-roche-raise-awareness-for-macular-disease-with-varjo-mixed-reality/

[25] International Agency for the Prevention of Blindness (IAPB). (2020). *IAPB Vision Atlas*.
Retrieved from https://visionatlas.iapb.org/

[26] R. Allen & L. K. Welch. (2023). *Leveraging virtual reality to foster engaging and interactive learning in healthcare*. *Diabetes*, 72(Suppl. 1), 499-P.
Retrieved from https://doi.org/10.2337/db23-499-P

[27] Z. Ai, B. Gupta, M. Rasmussen, Y. Lin, F. Dech, W. Panko, & J. Silverstein. (2000). *Simulation of eye diseases in a virtual environment*. In *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*.
Retrieved from https://ieeexplore.ieee.org/document/926803

[28] C. W. Ong, M. C. J. Tan, M. Lam, & V. T. C. Koh. (2021). *Applications of extended reality in ophthalmology: A systematic review*. *Journal of Medical Internet Research*.

Retrieved from
https://journals.lww.com/retinajournal/citation/2017/02000/simulation_training_in_vitreoretinal_surgery__a.26.aspx

[29] P. R. Jones, T. Somoskeöy, H. Chow-Wing-Bom, & D. P. Crabb. (2020). *Seeing other perspectives: Evaluating VR and AR for visual impairment simulation (OpenVisSim)*. *npj Digital Medicine*, 3(1).

Retrieved from https://www.nature.com/articles/s41746-020-0242-6

[30] EyeWiki. American Academy of Ophthalmology. (n.d.).
Retrieved from https://eyewiki.org/MainPage

[31] University of Utah. (n.d.). *Webvision: The Organization of the Retina and Visual System*.
Retrieved from https://webvision.med.utah.edu/